POWERSHELL

CHEAT SHEETS

A Complete Reference for PowerShell 7

Cmdlets · Pipeline · Objects · Operators · Functions · Error Handling · Hashtables · .NET & Classes · Files & JSON · Modules · Remoting · REST APIs

42 REFERENCE SHEETS

CONTENTS

GETTING STARTED

01 Introduction & Editions
02 Cmdlet Syntax & Discovery
03 Profiles & PSReadLine

OUTPUT & COMMENTS

04 Output & Streams
05 Comments & Comment-Based Help

VARIABLES & TYPES

06 Variables & Scope
07 Automatic & Preference Variables
08 Types & Casting

STRINGS

09 String Basics & Formatting
10 String Methods & Operators

OPERATORS

11 Arithmetic & Assignment Operators
12 Comparison & Matching Operators
13 Logical, Null & Special Operators

CONTROL FLOW

14 if & switch
15 Loops

THE PIPELINE

16 Pipeline Basics
17 Custom Objects & Calculated Properties
18 Formatting Output

ERROR HANDLING

19 Errors & ErrorAction
20 try / catch / finally & throw

FUNCTIONS

21 Functions & Script Blocks
22 Advanced Functions & Parameter Validation
23 Pipeline Input & ShouldProcess

COLLECTIONS

24 Arrays & Lists
25 Hashtables & Dictionaries

.NET & CLASSES

26 Working with .NET
27 Classes & Enums

FILES & DATA

28 File System & Items
29 Reading & Writing Files
30 JSON, CSV & XML

MODULES

31 Using Modules & the PowerShell Gallery
32 Writing Modules

ADVANCED TOPICS

33 Regular Expressions
34 Dates & Times
35 Jobs & Parallelism
36 Web Requests & REST APIs
37 Debugging & Testing

SYSTEM ADMINISTRATION

38 Processes, Services & Native Commands
39 Remoting & Credentials
40 Windows Management

QUICK REFERENCE

41 Quick Reference — Language
42 Quick Reference — Pipeline, Files & System

PS

Table of Contents

42 sheets · syntax · examples · notes

GETTING STARTED 01 – 03

01 Introduction & Editions | 02 Cmdlet Syntax & Discove… | 03 Profiles & PSReadLine

OUTPUT & COMMENTS 04 – 05

04 Output & Streams | 05 Comments & Comment-…

VARIABLES & TYPES 06 – 08

06 Variables & Scope | 07 Automatic & Preference … | 08 Types & Casting

STRINGS 09 – 10

09 String Basics & Formatting | 10 String Methods & Operat…

OPERATORS 11 – 13

11 Arithmetic & Assignment … | 12 Comparison & Matching … | 13 Logical, Null & Special Op…

CONTROL FLOW 14 – 15

14 if & switch | 15 Loops

THE PIPELINE 16 – 18

16 Pipeline Basics | 17 Custom Objects & Calcul… | 18 Formatting Output

ERROR HANDLING 19 – 20

19 Errors & ErrorAction | 20 try / catch / finally & throw

FUNCTIONS 21 – 23

21 Functions & Script Blocks | 22 Advanced Functions & Pa… | 23 Pipeline Input & ShouldPr…

COLLECTIONS 24 – 25

24 Arrays & Lists | 25 Hashtables & Dictionaries

.NET & CLASSES 26 – 27

26 Working with .NET | 27 Classes & Enums

FILES & DATA 28 – 30

28 File System & Items | 29 Reading & Writing Files | 30 JSON, CSV & XML

MODULES 31 – 32

31 Using Modules & the Pow… | 32 Writing Modules

ADVANCED TOPICS 33 – 37

33 Regular Expressions | 34 Dates & Times | 35 Jobs & Parallelism

36 Web Requests & REST APIs | 37 Debugging & Testing

SYSTEM ADMINISTRATION 38 – 40

38 Processes, Services & Na… | 39 Remoting & Credentials | 40 Windows Management

QUICK REFERENCE 41 – 42

41 Quick Reference — Langu… | 42 Quick Reference — Pipeli…

Introduction & Editions

OVERVIEW

PowerShell is a cross-platform shell and scripting language built on .NET. Its big idea is that commands pass **objects** down the pipeline, not text, so you filter and sort by property instead of parsing columns. There are two editions: **Windows PowerShell 5.1** (`powershell.exe`, .NET Framework, Windows only, maintenance mode) and **PowerShell 7** (`pwsh`, modern .NET, Windows/macOS/Linux). Both can be installed side by side.

NOTES

- Script files use `.ps1`; modules `.psm1`; manifests `.psd1`
- Commands are case-insensitive: `get-process` = `Get-Process`
- This set targets PowerShell 7.4+; 5.1 gaps are flagged as *5.1:*
- VS Code + the PowerShell extension is the standard editor (ISE is 5.1-only)

INSTALL, CHECK, RUN

```
# Install PowerShell 7
winget install --id Microsoft.PowerShell     # Windows
brew install powershell/tap/powershell       # macOS
pwsh                                         # start a session

$PSVersionTable.PSVersion     # 7.4.6 (Major.Minor.Patch)
$PSVersionTable.PSEdition     # Core   (5.1 shows Desktop)

# hello.ps1
#Requires -Version 7.0
param([string]$Name = 'World')
"Hello, $Name!"                # implicit output → Hello, World!

.\hello.ps1 -Name Bob          # Hello, Bob!  (.\ is required)
pwsh -File ./hello.ps1         # run from another shell
```

CODE EXPLAINED

- → `winget / brew` — official package installs; Linux uses apt/dnf repos or Snap
- → `$PSVersionTable` — hashtable of version, edition, OS details
- → `#Requires` — script refuses to run if the version/module/admin check fails
- → `param()` — script-level parameters, same syntax as functions
- → `.\hello.ps1` — the current folder is not on the path, so prefix `.\` or `./`

EDITIONS COMPARED

FEATURE	5.1	7.X
Executable	powershell.exe	pwsh
Runtime	.NET Framework	.NET 8+
Platforms	Windows	All
?? ?. a?b:c	✖	✔
-Parallel	✖	✔
Encoding	ANSI	UTF-8

EXECUTION POLICIES

POLICY	EFFECT
Restricted	no scripts (Windows client default)
RemoteSigned	local OK; downloaded must be signed
AllSigned	every script must be signed
Bypass	nothing blocked, no warnings
Unrestricted	runs all, warns on downloads

WAYS TO RUN CODE

FORM	USE
.\script.ps1	run in a child scope
. .\script.ps1	dot-source into current scope
& 'C:\My Dir\x.ps1'	call a quoted path
pwsh -NoProfile	skip profile scripts
pwsh -c '...'	short for -Command

⚠ COMMON MISTAKES

- Typing `script.ps1` without `.\` fails with "not recognized". PowerShell never runs scripts from the current folder implicitly, as a safety feature
- Assuming 5.1 and 7 share modules. 7 loads most 5.1 modules, but some Windows-only ones need `-UseWindowsPowerShell`
- Using `Set-ExecutionPolicy Unrestricted` machine-wide to fix one script. Use `-Scope CurrentUser RemoteSigned` or `Unblock-File`

✓ TIPS & BEST PRACTICES

- Check `$PSVersionTable` first when an example doesn't work. Most "broken" snippets are 7-only features run in 5.1
- Set `RemoteSigned` per user: `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`
- Add `#Requires -Version 7.0` to scripts that use 7-only syntax so they fail with a clear message

QUICK SUMMARY

- `pwsh` — PowerShell 7, cross-platform; `powershell` is 5.1
- `$PSVersionTable` — version and edition info
- `.\script.ps1` — run a script; needs a path prefix
- `Set-ExecutionPolicy` — controls whether scripts may run (Windows)

Cmdlet Syntax & Discovery

OVERVIEW

Cmdlets are named **Verb-Noun** using a fixed list of approved verbs (`Get`, `Set`, `New`, `Remove`…), so you can guess names. Three commands let you learn anything from inside the shell: `Get-Command` finds commands, `Get-Help` explains them, and `Get-Member` shows what properties and methods an object has.

NOTES

- Parameters start with `-` and can be shortened while unique: `-Na` = `-Name`
- Switch parameters are on/off flags: `-Recurse`, `-Force`
- Run `Update-Help` once (as admin in 5.1) to download full help
- Tab completes commands, parameters, paths and even property names

DISCOVER, LEARN, INSPECT

```
Get-Command *service*              # anything with "service" in the name
Get-Command -Verb Get -Noun *Item* # Get-Item, Get-ChildItem, …
Get-Command -Module Microsoft.PowerShell.Utility

Get-Help Get-ChildItem -Examples   # just the examples
Get-Help Get-ChildItem -Parameter Filter
Get-Help about_Operators           # concept topics start with about_
Get-ChildItem -?                   # quick syntax summary

Get-Process | Get-Member           # TypeName + properties + methods
Get-Process | Get-Member -MemberType Property

Get-ChildItem -Path C:\Temp -Filter *.log -Recurse   # named
Get-ChildItem C:\Temp *.log -Recurse                 # positional
Get-Alias ls, gci, dir             # all → Get-ChildItem
Get-Verb | Where-Object Group -eq Lifecycle
```

CODE EXPLAINED

- → `Get-Command` — search by wildcard, `-Verb`, `-Noun`, `-Module`
- → `-Examples` — fastest way to learn a cmdlet; also `-Full`, `-Online`
- → `about_*` — conceptual help: `about_Splatting`, `about_Scopes` …
- → `Get-Member` — what an object *is*: type name, properties, methods
- → `positional` — `-Path` and `-Filter` are positions 0 and 1 on `Get-ChildItem`
- → `Get-Alias` — maps short names back to cmdlets

COMMON VERBS

VERB	MEANING
`Get / Set`	read / change
`New / Remove`	create / delete
`Add / Clear`	append / empty
`Start / Stop`	begin / end
`Import / Export`	from / to a file
`ConvertTo / From`	change format
`Invoke`	run an action

PARAMETER FORMS

FORM	EXAMPLE
`Named`	-Name notepad
`Positional`	Get-Process notepad
`Switch`	-Recurse (= -Recurse:$true)
`Array`	-Name a, b, c
`Wildcard`	-Name note*
`Abbreviated`	-Recu

BUILT-IN ALIASES

ALIAS	CMDLET
`ls / gci / dir`	Get-ChildItem
`cd / sl`	Set-Location
`cat / gc`	Get-Content
`? / where`	Where-Object
`% / foreach`	ForEach-Object
`select`	Select-Object
`gm`	Get-Member

⚠ COMMON MISTAKES

- Reading `ls` output as text. It is objects; use `Get-Member` to see the real property names before filtering
- Using aliases like `%`, `?` or `ls` in scripts. They hurt readability and `ls` / `cat` aren't aliases on Linux/macOS pwsh
- Skipping `Update-Help`. Without it `Get-Help` shows only syntax stubs

✓ TIPS & BEST PRACTICES

- `Get-Help name -Online` opens the current docs page in your browser
- Pipe any result to `Get-Member` before writing `Where-Object` or `Select-Object`
- Use `Show-Command Get-ChildItem` (Windows) for a form-based parameter picker

QUICK SUMMARY

- `Verb-Noun` — cmdlet naming; see `Get-Verb`
- `Get-Command` — find commands by name, verb, noun, module
- `Get-Help -Examples` — learn by example
- `Get-Member` — inspect an object's properties and methods

Profiles & PSReadLine

OVERVIEW

A **profile** is a script that runs every time PowerShell starts, where you put aliases, functions, modules and your prompt. **PSReadLine** is the module that powers the interactive line editor: syntax colors, history search, predictive IntelliSense and key bindings. Both are per-user and per-host, and both make the shell feel like your own.

NOTES

- `$PROFILE` points to CurrentUserCurrentHost; it may not exist yet
- 5.1 and 7 use different profile folders
- Start with `pwsh -NoProfile` to rule out profile problems
- Keep profiles fast; slow imports delay every new tab

PROFILE SETUP

```
$PROFILE                          # path to your profile script
Test-Path $PROFILE                # False on a fresh install
New-Item $PROFILE -ItemType File -Force
code $PROFILE                     # edit in VS Code
. $PROFILE                        # reload without restarting

# --- inside the profile ---
Set-Alias ll Get-ChildItem
function prompt { "PS $(Split-Path -Leaf $PWD)> " }

Set-PSReadLineOption -PredictionSource HistoryAndPlugin
Set-PSReadLineOption -PredictionViewStyle ListView
Set-PSReadLineOption -EditMode Windows      # or Emacs / Vi
Set-PSReadLineKeyHandler -Key Tab -Function MenuComplete
Set-PSReadLineKeyHandler -Key UpArrow -Function HistorySearchBackward

oh-my-posh init pwsh | Invoke-Expression    # themed prompt
```

CODE EXPLAINED

- → `$PROFILE` — path string with extra properties for all four profiles
- → `New-Item -Force` — creates the file and any missing folders
- → `. $PROFILE` — dot-sources it so changes apply to this session
- → `function prompt` — whatever it returns becomes your prompt
- → `PredictionSource` — ghost-text suggestions from history (and plugins)
- → `MenuComplete` — Tab shows a selectable menu, like zsh

THE FOUR PROFILES

$PROFILE.	APPLIES TO
`CurrentUserCurrentHost`	you, this host (default)
`CurrentUserAllHosts`	you, console + VS Code
`AllUsersCurrentHost`	everyone, this host
`AllUsersAllHosts`	everyone, every host

DEFAULT KEYS

KEY	ACTION
`Ctrl+R`	reverse history search
`F2`	toggle inline / list predictions
`Ctrl+Space`	show completion menu
`Alt+.`	insert last argument
`Ctrl+L`	clear screen
`Esc`	clear the line

HISTORY COMMANDS

COMMAND	USE
`h`	Get-History: this session
`r 5`	Invoke-History: rerun #5
`#ls<Tab>`	complete from history
`Clear-History`	clear session list only

⚠ COMMON MISTAKES

- Editing the 5.1 profile and wondering why pwsh ignores it. Each edition has its own `$PROFILE` path
- Putting heavy `Import-Module` calls in the profile. Modules autoload on first use, so most imports are unnecessary
- Assuming `Clear-History` removes saved history. PSReadLine keeps its own file at `HistorySavePath`

✓ TIPS & BEST PRACTICES

- Use `$PROFILE.CurrentUserAllHosts` for settings you want in both the terminal and VS Code
- Measure startup with `pwsh -NoProfile -Command "Measure-Command { . $PROFILE }"`
- Run `Get-PSReadLineKeyHandler` to list every binding and its key

QUICK SUMMARY

- `$PROFILE` — startup script path; create it with `New-Item -Force`
- `function prompt` — define a custom prompt
- `Set-PSReadLineOption` — predictions, edit mode, colors
- `Ctrl+R` / `Get-History` — find and rerun past commands

Output & Streams

OVERVIEW

Anything a command or expression produces and doesn't capture goes to the **success stream** (stream 1), so a bare string or variable on its own line *is* output. `Write-Host` writes straight to the screen and cannot be piped or captured into a variable. PowerShell has six numbered streams, each with its own cmdlet, and all can be redirected.

NOTES

- Implicit output and `Write-Output` are the same thing
- Use `Write-Host` only for colored, display-only messages
- Verbose/Debug are hidden unless you pass `-Verbose` / `-Debug`
- Function output = everything not captured, not just `return`

WRITING OUTPUT

```
"Hello"                          # success stream → Hello
Write-Output "Hello"             # same as above
$x = Write-Output "Hi"           # captured: $x = 'Hi'
$y = Write-Host "Hi"             # prints Hi; $y is $null

Write-Host "Done" -ForegroundColor Green
Write-Verbose "Step 2"  -Verbose # VERBOSE: Step 2
Write-Warning "Low disk"         # WARNING: Low disk
Write-Error "Not found"          # red error, non-terminating

.\build.ps1 > out.txt            # stream 1 to file
.\build.ps1 *> all.txt           # every stream to file

New-Item x.txt | Out-Null        # discard output
$null = New-Item y.txt           # faster way to discard
```

CODE EXPLAINED

- → `implicit output` — an uncaptured value goes down the pipeline
- → `Write-Host` — writes to stream 6 (Information) for display; not captured by `$var =`
- → `2>&1` — redirect stream 2 into stream 1 so errors can be captured
- → `$null =` — discard; also `[void](...)` and `| Out-Null`

THE SIX STREAMS

#	STREAM	CMDLET
1	Success	Write-Output
2	Error	Write-Error
3	Warning	Write-Warning
4	Verbose	Write-Verbose
5	Debug	Write-Debug
6	Information	Write-Information

REDIRECTION OPERATORS

OPERATOR	EFFECT
`> / >>`	stream 1 to file (overwrite / append)
`2> / 2>>`	errors to file
`*>`	all streams
`2>&1`	errors into output
`2>$null`	discard errors

OUT- CMDLETS

CMDLET	SENDS TO
`Out-File`	file (–Append)
`Out-String`	one string
`Out-Null`	nowhere
`Out-Host -Paging`	screen, paged
`Out-GridView`	sortable window (Windows)

⚠ COMMON MISTAKES

- Using `Write-Host` in a function that should return data. The caller gets nothing back; output with a bare value or `Write-Output`
- Stray output: `$list.Add(1)` on an ArrayList returns an index that leaks into the function result. Discard with `$null =`
- Expecting `Write-Error` to stop the script. It's non-terminating; use `throw` or `-ErrorAction Stop`

✓ TIPS & BEST PRACTICES

- Use `Write-Verbose` for progress notes; users opt in with `-Verbose`
- Prefer `$null = …` over `| Out-Null` in loops. It skips the pipeline and is much faster
- Use `Write-Progress -Activity 'Copy' -PercentComplete 40` for long jobs

QUICK SUMMARY

- `"text"` — implicit output to stream 1
- `Write-Host` — display only, can't be captured
- `Write-Verbose/Warning/Error` — streams 4, 3, 2
- `2>&1` / `*>` — merge or redirect streams

Comments & Comment-Based Help

OVERVIEW

Use # for single-line comments and <# … #> for blocks. A block comment placed at the top of a function or script with special **help keywords** becomes real help: `Get-Help` shows it exactly like a built-in cmdlet's help, and VS Code shows it on hover.

NOTES

- Help block goes just inside the function body or just above `function`
- Keywords start with a dot and are case-insensitive
- `.PARAMETER` names must match the actual parameters
- `#region` / `#endregion` create foldable sections in VS Code

COMMENT-BASED HELP

```powershell
function Get-DiskFree {
    <#
    .SYNOPSIS
        Shows free space per drive.
    .DESCRIPTION
        Returns one object per fixed drive with size and free GB.
    .PARAMETER MinFreeGB
        Only show drives with less free space than this.
    .EXAMPLE
        Get-DiskFree -MinFreeGB 20
    #>
    param([int]$MinFreeGB = [int]::MaxValue)
    Get-CimInstance Win32_LogicalDisk -Filter 'DriveType=3' |
        Where-Object { $_.FreeSpace / 1GB -lt $MinFreeGB }
}
Get-Help Get-DiskFree -Examples
```

CODE EXPLAINED

- → `<# … #>` — block comment; can span many lines
- → `.SYNOPSIS` — one-line summary shown in `Get-Help` lists
- → `.PARAMETER name` — one block per parameter
- → `.EXAMPLE` — code line, then a description line
- → `# TODO` — ordinary comment; VS Code can highlight TODOs
- → `Get-Help -Examples` — works on your function right away

HELP KEYWORDS

KEYWORD	PURPOSE
`.SYNOPSIS`	short summary
`.DESCRIPTION`	full explanation
`.PARAMETER x`	describe parameter x
`.EXAMPLE`	usage + explanation
`.NOTES`	extra info
`.LINK`	related URL or command

COMMENT SYNTAX

SYNTAX	USE
`# text`	single line
`<# … #>`	block / help
`#region Name`	start fold
`#endregion`	end fold
`#Requires`	not a comment: a directive
`#!/usr/bin/env pwsh`	shebang line (Linux/macOS)

GET-HELP SWITCHES

SWITCH	SHOWS
`-Detailed`	+ parameters
`-Full`	everything
`-Examples`	examples only
`-Parameter X`	one parameter
`-ShowWindow`	popup (Windows)

⚠ COMMON MISTAKES

- Leaving two or more blank lines between a help block and `function`. Help placed above a function is ignored unless it sits right above the keyword
- Misspelling a `.PARAMETER` name. The entry silently doesn't show; names must match exactly
- Using `#` comments for help. Only `<# #>` blocks with dot-keywords are parsed as help

✓ TIPS & BEST PRACTICES

- Put help *inside* the function, right after `{`, so it travels with the code when moved
- Write at least `.SYNOPSIS` and one `.EXAMPLE`; it's the part people actually read
- Use `#region` to group a long script into sections you can fold

QUICK SUMMARY

- `#` — single-line comment
- `<# #>` — block comment
- `.SYNOPSIS` / `.EXAMPLE` — comment-based help keywords
- `#region` — foldable section

Variables & Scope

OVERVIEW

Variables start with $, need no declaration, and can hold any object. Names are case-insensitive. PowerShell uses **dynamic scope**: a child scope (function, script block, script) can *read* its parent's variables, but assigning creates a new local copy unless you name a scope with a modifier like `$script:`.

NOTES

- `$Name` and `$name` are the same variable
- Unset variables are `$null` (error only under `Set-StrictMode`)
- Environment variables live on the `env:` drive
- Variables persist for the session; scripts get their own scope

VARIABLES IN ACTION

```
$count = 5                      # create / assign
$a, $b = 1, 2                   # multiple assignment
$a, $b = $b, $a                 # swap → a=2, b=1
${my var} = 'ok'                # braces allow spaces/symbols

New-Variable -Name Pi2 -Value 6.283 -Option ReadOnly
Set-Variable  -Name Max -Value 10 -Option Constant

$env:APP_MODE = 'dev'           # set for this process
$env:PATH -split [IO.Path]::PathSeparator   # one path per line

$total = 0
function Add-One { $total = $total + 1 }       # local copy
function Add-Two { $script:total += 2 }        # script scope
Add-One; Add-Two; $total                       # 2

Get-Variable c*; Test-Path variable:count      # True
Remove-Variable count
```

CODE EXPLAINED

- → `$a, $b = 1, 2` — spreads values across variables; extras go to the last
- → `${my var}` — needed for names with spaces, dashes, or `:`
- → `ReadOnly` — change with `-Force`; `Constant` can never change
- → `$env:` — process-level; persists only while the session lives
- → `$script:total` — assigns in the script scope instead of the function
- → `variable:` — the variable drive, usable with `Test-Path`

SCOPE MODIFIERS

MODIFIER	MEANING
`$global:x`	session-wide
`$script:x`	current script file
`$local:x`	current scope (default)
`$private:x`	not visible to children
`$using:x`	local value into remote / -Parallel
`$env:x`	environment variable

VARIABLE CMDLETS

CMDLET	USE
`Get-Variable`	list / find variables
`New-Variable`	create with options
`Set-Variable`	set, incl. –Scope
`Clear-Variable`	set to $null
`Remove-Variable`	delete entirely
`Get-ChildItem env:`	list env vars

NAMING CONVENTIONS

STYLE	EXAMPLE
`camelCase`	$fileCount (locals)
`PascalCase`	$UserName (params)
`Avoid`	$input, $args, $_ (automatic)
`Constants`	$MaxRetries

⚠ COMMON MISTAKES

- Expecting a function to change an outer variable by assignment. It creates a local copy; return the value or use `$script:`
- Naming a variable `$input`, `$args` or `$PSItem`. These are automatic variables and get overwritten
- Setting `$env:PATH` and expecting it to persist. It lasts only for the process; use `[Environment]::SetEnvironmentVariable` with `'User'`

✓ TIPS & BEST PRACTICES

- Return values from functions instead of reaching into `$global:`
- Turn on `Set-StrictMode -Version Latest` in scripts to catch typos in variable names
- Use `$PSDefaultParameterValues` for session-wide parameter defaults

QUICK SUMMARY

- `$x = value` — create or reassign; no declaration
- `$script:` / `$global:` — write to an outer scope
- `$env:NAME` — read or set environment variables
- `New-Variable -Option` — ReadOnly or Constant

Automatic & Preference Variables

OVERVIEW

Automatic variables are created and updated by PowerShell itself: the current pipeline object, the last command's success, the script's folder. **Preference variables** are settings you can change, such as how errors or verbose messages are handled. Don't assign to automatic variables; do tune preference variables, ideally only inside a script or function.

NOTES

- `$_` and `$PSItem` are identical; `$_` is more common
- `$?` is `$true` if the last command fully succeeded
- `$LASTEXITCODE` is only set by native programs and scripts' `exit`
- `$PSScriptRoot` is empty at the interactive prompt

COMMON AUTOMATIC VARIABLES

```powershell
Get-Process | Where-Object { $_.CPU -gt 100 }   # current object

git status; $LASTEXITCODE        # 0 = success for native exes
Get-Item nope.txt; $?            # False

$PSScriptRoot                    # C:\scripts  (folder of this .ps1)
$PSCommandPath                   # C:\scripts\deploy.ps1
Join-Path $PSScriptRoot 'config.json'

'2026-10-06' -match '(\d{4})-(\d\d)'
$Matches[0]; $Matches[1]         # 2026-10 / 2026

$ErrorActionPreference = 'Stop'  # make errors terminating
$VerbosePreference = 'Continue'  # show Write-Verbose output
$Error[0]                        # most recent error
```

CODE EXPLAINED

- → `$_` — set inside `Where-Object`, `ForEach-Object`, `switch`, `catch`
- → `$LASTEXITCODE` — check after `git`, `npm`, `robocopy` and other exes
- → `$PSScriptRoot` — reliable way to find files next to the script
- → `$Matches` — hashtable filled by a successful scalar `-match`
- → `Preference` — values: `Stop`, `Continue`, `SilentlyContinue`, `Inquire`

AUTOMATIC VARIABLES

VARIABLE	HOLDS
`$true $false $null`	constants
`$HOME / $PWD`	home / current dir
`$PID`	this session's process ID
`$Error`	list of recent errors
`$input`	pipeline enumerator in functions
`$IsWindows`	platform flag (7+)

PREFERENCE VARIABLES

VARIABLE	DEFAULT
`$ErrorActionPreference`	Continue
`$WarningPreference`	Continue
`$VerbosePreference`	SilentlyContinue
`$DebugPreference`	SilentlyContinue
`$ProgressPreference`	Continue
`$ConfirmPreference`	High

HANDY EXTRAS

VARIABLE	USE
`$args`	unnamed function args
`$OFS`	array join char
`$MyInvocation`	caller info
`$Host`	console info
`$PSStyle`	ANSI colors (7.2+)

⚠ COMMON MISTAKES

- Reading `$?` after another command ran. Even an `echo` resets it; check it on the very next line or capture it
- Using `$PSScriptRoot` in code run by selection (F8). It's empty outside a running script file
- Assuming cmdlets set `$LASTEXITCODE`. Only native programs and `exit N` do; use `$?` or `try/catch` for cmdlets

✓ TIPS & BEST PRACTICES

- Set `$ProgressPreference = 'SilentlyContinue'` to speed up `Invoke-WebRequest` in 5.1
- Change preferences inside a function; they revert automatically when it returns
- Use `$PSDefaultParameterValues` to set session-wide parameter defaults

QUICK SUMMARY

- `$_` / `$PSItem` — current pipeline object
- `$?` / `$LASTEXITCODE` — success of last cmdlet / native exe
- `$PSScriptRoot` — folder of the running script
- `$ErrorActionPreference` — default error behavior

Types & Casting

OVERVIEW

Every value is a .NET object with a type. PowerShell converts types for you, and the **left operand decides**: `'5' + 3` is the string `'53'`, but `5 + '3'` is 8. Cast explicitly with `[type]`, test with `-is`, and convert safely with `-as`, which returns `$null` instead of throwing.

NOTES

- `[int]` casts round to even: `[int]2.5` → 2, `[int]3.5` → 4
- A typed variable `[int]$n` stays an int on every later assignment
- Any non-empty string is `$true`: `[bool]'False'` → True
- Type names are case-insensitive and `System.` is optional

CASTING & CONVERSION

```
[int]'42'                # 42
[int]'4.6'               # 5
[string]42               # '42'
[datetime]'2026-10-06'   # Tuesday, October 6, 2026 12:00:00 AM
[bool]0; [bool]'0'       # False; True  (string is non-empty)

'5' + 3                  # 53   (string on the left)
5 + '3'                  # 8    (number on the left)

'abc' -as [int]          # $null, no error
42 -is [int]             # True
(42).GetType().Name      # Int32

[int]$n = 10
$n = '20'                # converted → 20
$n = 'abc'               # error: cannot convert
```

CODE EXPLAINED

- → `[int]'4.6'` — string → number parses then rounds
- → `[datetime]` — parses culture-invariant formats like `yyyy-MM-dd`
- → `'5' + 3` — left side is a string, so `+` concatenates
- → `-as` — safe conversion; test the result for `$null`
- → `.GetType()` — returns the type; `.Name` / `.FullName` for text
- → `[int]$n` — type constraint applies to the variable itself

TYPE ACCELERATORS

ACCELERATOR	.NET TYPE
`[int]` / `[long]`	Int32 / Int64
`[double]` / `[decimal]`	Double / Decimal
`[string]` / `[char]`	String / Char
`[bool]`	Boolean
`[datetime]` / `[timespan]`	DateTime / TimeSpan
`[hashtable]` / `[array]`	Hashtable / Object[]

NUMERIC LITERALS

LITERAL	VALUE
`1kb 1mb 1gb`	1024 1048576 1073741824
`0xFF`	255
`0b1010`	10 (7.0+)
`1e3`	1000 (double)
`42L`	Int64
`1.5d`	Decimal

MORE ACCELERATORS

ACCELERATOR	USE
`[regex]`	compiled pattern
`[xml]`	parse XML text
`[version]`	'1.10' -gt '1.9' properly
`[guid]`	[guid]::NewGuid()
`[ipaddress]`	validate an IP
`[pscustomobject]`	custom object

⚠ COMMON MISTAKES

- Comparing versions as strings. `'1.10' -gt '1.9'` is False; cast with `[version]` first
- Casting user input with `[int]` and letting it throw. Use `-as [int]` or `[int]::TryParse()` and check the result
- Expecting `[bool]'false'` to be False. Strings are true when non-empty; compare text instead

✓ TIPS & BEST PRACTICES

- Put the value whose type you want on the left: `[int]$a + $b` forces numeric addition
- Use `[decimal]` for money; `[double]` has binary rounding errors (0.1 + 0.2)
- Run `Get-Member` to see the type name at the top of the output

QUICK SUMMARY

- `[type]value` — explicit cast
- `-as` / `-is` — safe convert / type test
- Left operand decides — `'5'+3` vs `5+'3'`
- `1kb`, `0xFF`, `42L` — numeric literal shortcuts

String Basics & Formatting

OVERVIEW

Single-quoted strings are **literal**. Double-quoted strings **expand** variables and `$( )` subexpressions. The escape character is the **backtick** (`` ` ``), not backslash. For layout and number formatting, use the `-f` format operator with .NET format strings.

NOTES

- Strings are immutable; methods return new strings
- Use `$( )` for properties: `"$($u.Name)"`
- Double a quote to escape it: `'It''s'`, `"Say ""hi"""`
- Here-string markers must be alone on their lines

QUOTING & FORMATTING

```
$name = 'Ada'; $n = 3
'Hi $name'                    # Hi $name   (literal)
"Hi $name"                    # Hi Ada
"Items: $($n * 2)"            # Items: 6
"User: $($user.Name)"         # property needs $( )
"Tab`there, newline`nnext"    # backtick escapes
"Cost: `$5"                   # Cost: $5

$body = @"
Dear $name,
Your total is $($n * 10).
"@                            # expanding here-string

'{0} has {1} items' -f $name, $n      # Ada has 3 items
'{0:N2}' -f 1234.5678                 # 1,234.57
'{0:P0}' -f 0.256                     # 26%
'[{0,-8}][{1,6}]' -f 'left', 'right' # [left    ][ right]
(42).ToString('D5')                   # 00042
```

CODE EXPLAINED

- → `'…'` — no expansion at all; safest for regex and paths
- → `$( )` — runs any expression inside a double-quoted string
- → `` `$ `` — escapes the dollar so it prints literally
- → `@" "@` — multi-line; `@' '@` is the literal version
- → `-f` — `{index:format}` placeholders, values on the right
- → `{0,-8}` — width 8, left-aligned; positive = right-aligned

BACKTICK ESCAPES

ESCAPE	RESULT
`` `n ``	newline
`` `t ``	tab
`` `r ``	carriage return
`` `0 ``	null char
`` `e ``	ESC for ANSI (7+)
`` `u{1F600} ``	Unicode (7+)

FORMAT SPECIFIERS

SPEC	1234.5 →
`{0:N2}`	1,234.50
`{0:F1}`	1234.5
`{0:C}`	$1,234.50
`{0:E2}`	1.23E+003
`{0:X}`	4D2 (int 1234)
`{0:D6}`	001234 (int)

DATE FORMATS

SPEC	RESULT
`{0:yyyy-MM-dd}`	2026-10-06
`{0:HH:mm:ss}`	14:05:09
`{0:ddd}`	Tue
`{0:MMMM}`	October
`{0:s}`	2026-10-06T14:05:09
`{0:o}`	round-trip ISO

⚠ COMMON MISTAKES

- Writing `"$user.Name"`. Only `$user` expands, then `.Name` is literal text; use `"$($user.Name)"`
- Using backslash escapes like `\n`. PowerShell's escape is the backtick; `\n` stays as two characters
- Indenting the closing `"@`. It must start at column 1 or the string never ends

✓ TIPS & BEST PRACTICES

- Default to single quotes; switch to double only when you need expansion
- Use `-f` instead of string concatenation for tables of output
- Format currency with a culture: `$n.ToString('C', [cultureinfo]'en-GB')`

QUICK SUMMARY

- `'…'` literal vs `"…"` expanding
- `$( )` — embed expressions and properties
- `` ` `` — escape character (`` `n ``, `` `t ``, `` `$ ``)
- `-f` — `'{0:N2}' -f $x` formatting

String Methods & Operators

-split / .Split() · -join · -replace / .Replace() · .Trim() · .Substring() · IsNullOrWhiteSpace

OVERVIEW

Strings have both **operators** (`-split`, `-join`, `-replace`) and .NET **methods** (`.Split()`, `.Replace()`). The key difference: operators are **case-insensitive and use regex**; methods are **case-sensitive and literal**. Pick based on whether you need a pattern.

NOTES

- `-split '.'` splits on *every* character (regex). Use `'\.'`
- Methods are case-sensitive: `'Hi'.Contains('hi')` → False
- `-replace` supports `$1` capture groups in the replacement
- Indexes are 0-based; `$s[-1]` is the last character

SPLITTING, JOINING, REPLACING

```
$s = '  Hello, World  '
$s.Trim()                          # 'Hello, World'

'a,b,,c' -split ','                # a b '' c
'a1b22c' -split '\d+'              # a b c  (regex)
'one two' -split ' ', 2            # max 2 pieces
'a.b.c'.Split('.')                 # a b c  (literal)
'x','y','z' -join '-'              # x-y-z

'Hello' -replace 'l', 'L'          # HeLLo
'2026-10-06' -replace '(\d+)-(\d+)-(\d+)', '$3/$2/$1'  # 06/10/2026
'a.b'.Replace('.', '!')            # a!b

'PowerShell'.Substring(0, 5)       # Power
'PowerShell'.IndexOf('Shell')      # 5
'PowerShell'.StartsWith('power', 'OrdinalIgnoreCase')  # True
[string]::IsNullOrWhiteSpace('  ')  # True
```

CODE EXPLAINED

- → `.Trim()` — removes whitespace, or the characters you pass
- → `-split ','` — regex split; empty strings are kept
- → `-split ' ', 2` — limit the number of pieces
- → `'$3/$2/$1'` — single quotes so PowerShell doesn't expand `$3`
- → `.Replace()` — literal and case-sensitive, no regex escaping needed
- → `'OrdinalIgnoreCase'` — string auto-converts to `[StringComparison]`

STRING OPERATORS

OPERATOR	EXAMPLE → RESULT
`-split`	'a b' -split ' ' → a, b
`-join`	1,2 -join '+' → 1+2
`-replace`	'aa' -replace 'a','b' → bb
`-like`	'file.txt' -like '*.txt' → True
`-match`	'abc' -match 'b' → True
`*`	'-' * 5 → -----

COMMON METHODS

METHOD	EXAMPLE → RESULT
`.ToLower()`	'Hi'.ToLower() → hi
`.Contains()`	'abc'.Contains('b') → True
`.EndsWith()`	'a.ps1'.EndsWith('.ps1') → True
`.PadLeft()`	'7'.PadLeft(3,'0') → 007
`.Insert()`	'ac'.Insert(1,'b') → abc
`.Length`	'abc'.Length → 3

OPERATOR VS METHOD

NEED	USE
`Case-insensitive`	-replace, -split, -like
`Literal text`	.Replace(), .Split()
`Regex pattern`	-replace, -split, -match
`Escape for regex`	[regex]::Escape($s)

⚠ COMMON MISTAKES

- Splitting on `.`, `|` or `$` with `-split`. These are regex metacharacters; escape them or use `.Split()`
- Using double quotes around a replacement with `$1`. PowerShell expands it to empty first; use single quotes
- Calling methods on `$null`. `$x.Trim()` throws if `$x` is null; test with `IsNullOrWhiteSpace` first

✓ TIPS & BEST PRACTICES

- Use `-split '\r?\n'` to split text into lines
- Use `[regex]::Escape($userText)` before putting user input into a pattern
- For case-insensitive substring tests use `$s -like '*word*'`

QUICK SUMMARY

- `-split` / `-join` — break apart / glue together
- `-replace` regex vs `.Replace()` literal
- `.Trim()`, `.Substring()`, `.IndexOf()` — .NET methods
- `[string]::IsNullOrWhiteSpace()` — null-safe empty check

Arithmetic & Assignment Operators

OVERVIEW

Arithmetic works as expected on numbers, but + and * also work on strings and arrays. Division returns a **double** unless the result is exact, so `7 / 2` is `3.5`. There is no exponent operator; use `[math]::Pow()`. Bitwise operators are words: `-band`, `-bor`, `-shl`.

NOTES

- `7 / 2` → 3.5; `6 / 2` → 3 (Int32)
- `[int]` rounds half to even; use `[math]::Floor()` to drop decimals
- The range `..` works on integers and (7+) characters
- `++` / `--` exist as prefix and postfix

ARITHMETIC IN PRACTICE

```
7 / 2                    # 3.5
7 % 2                    # 1
[math]::Floor(7 / 2)     # 3
[math]::Round(2.567, 2)  # 2.57
[math]::Pow(2, 10)       # 1024
'ab' * 3                 # ababab
@(1, 2) * 2              # 1 2 1 2
'a' + 1                  # a1

$i = 5; $i++; $i         # 6
$total = 10; $total += 5 # 15
$s = 'Log'; $s += '.txt' # Log.txt
$list = @(); $list += 4  # array now has 1 item

1..5                     # 1 2 3 4 5
'a'..'e'                 # a b c d e (7+)
5 -band 3; 5 -bor 3      # 1; 7
```

CODE EXPLAINED

→ `7 / 2` — division never truncates; floor it yourself

→ `[math]::Round()` — round half to even by default; pass `'AwayFromZero'` for school rounding

→ `'ab' * 3` — string repetition; handy for `'-' * 40` separators

→ `+=` — works on numbers, strings and arrays (arrays are copied)

→ `1..5` — inclusive range; also counts down: `5..1`

ARITHMETIC

OP	EXAMPLE	RESULT
+	5 + 2	7
-	5 - 2	3
*	5 * 2	10
/	5 / 2	2.5
%	5 % 2	1
-	-5	negation

ASSIGNMENT

OP	SAME AS
`$a += 2`	$a = $a + 2
`$a -= 2`	$a = $a - 2
`$a *= 2`	$a = $a * 2
`$a /= 2`	$a = $a / 2
`$a %= 2`	$a = $a % 2
`$a ??= 0`	set if $null (7+)

BITWISE (5 = 0101, 3 = 0011)

OP	EXAMPLE	RESULT
-band	5 -band 3	1 (0001)
-bor	5 -bor 3	7 (0111)
-bxor	5 -bxor 3	6 (0110)
-bnot	-bnot 0	-1
-shl	1 -shl 4	16
-shr	16 -shr 2	4

⚠ COMMON MISTAKES

- Using `**` or `^` for powers. Neither exists (`^` is not XOR either); use `[math]::Pow()`
- Growing a big array with `+=` in a loop. Each add copies the whole array; use a `List[object]` or loop output
- Expecting `[int](7/2)` to be 3. It rounds to even → 4; use `[math]::Floor()` or `[math]::Truncate()`

✓ TIPS & BEST PRACTICES

- Use `[math]::Round($x, 2, 'AwayFromZero')` for invoice-style rounding
- Size math reads well with literals: `$file.Length / 1MB`
- Use `[int][math]::Ceiling($n / $pageSize)` to count pages

QUICK SUMMARY

- `+ - * / %` — division returns double when inexact
- `+=` / `++` — compound assignment and increment
- `[math]::` — Pow, Round, Floor, Ceiling, Abs, Max
- `-band -bor -bxor -shl` — bitwise ops

Comparison & Matching Operators

OVERVIEW

Comparison operators are words, not symbols (> is redirection!). They are **case-insensitive by default**; prefix with c for case-sensitive (-ceq). When the left side is an **array**, the operator acts as a **filter** and returns the matching elements instead of True/False.

NOTES

- `-like` uses wildcards; `-match` uses regex
- `-contains` tests collection membership, not substrings
- Put `$null` on the left: `$null -eq $x`
- Right side is converted to the left side's type

COMPARING VALUES

```
5 -eq 5                    # True
'abc' -eq 'ABC'            # True  (case-insensitive)
'abc' -ceq 'ABC'           # False
10 -gt 9                   # True
'10' -gt '9'               # False (string comparison)

'report.pdf' -like '*.pdf' # True
'v1.2.3' -match '^v(\d+)'  # True; $Matches[1] = '1'
'abc' -notmatch '\d'       # True

1, 2, 3, 2 -eq 2           # 2 2   (filter, not bool)
1, 5, 9 -gt 4              # 5 9
'a', 'b' -contains 'b'     # True
'b' -in 'a', 'b'           # True
'hello' -contains 'ell'    # False (not a substring test!)

$null -eq $items           # safe null test
```

CODE EXPLAINED

→ `-eq` — equality; `-ne` for not equal

→ `'10' -gt '9'` — string on the left → alphabetical compare

→ `-match` — regex; fills `$Matches` on scalar input

→ `1,2,3,2 -eq 2` — array on left → returns matching items

→ `-contains / -in` — same test, collection on opposite sides

→ `$null -eq` — with an array on the left you'd get a filter instead

COMPARISON

OP	MEANING
`-eq / -ne`	equal / not equal
`-gt / -ge`	greater / or equal
`-lt / -le`	less / or equal
`-ceq, -cgt…`	case-sensitive
`-ieq, -igt…`	explicitly insensitive
`-is / -isnot`	type test

MATCHING

OP	PATTERN TYPE
`-like`	wildcard: * ? [a-z]
`-notlike`	wildcard, negated
`-match`	regex, sets $Matches
`-notmatch`	regex, negated
`-cmatch`	regex, case-sensitive
`-replace`	regex substitution

CONTAINMENT

EXPRESSION	RESULT
`1,2,3 -contains 2`	True
`1,2,3 -notcontains 5`	True
`2 -in 1,2,3`	True
`5 -notin 1,2,3`	True
`'a*' -like 'a*'`	True

⚠ COMMON MISTAKES

- Using `>` or `==`. `>` redirects output to a file named after the right side; use `-gt` and `-eq`
- Writing `if ($list -eq $null)`. With an array on the left this filters and is usually falsy; write `$null -eq $list`
- Using `-contains` for substrings. It checks whole items; use `-like '*x*'` or `.Contains()`

✓ TIPS & BEST PRACTICES

- Use `-in` in `Where-Object` for allow-lists: `Where-Object Status -in 'Running','Paused'`
- Cast both sides when types are unclear: `[int]$a -gt [int]$b`
- Anchor regexes (`^…$`) when you need a whole-string match

QUICK SUMMARY

- `-eq -ne -gt -lt -ge -le` — compare; `c` prefix = case-sensitive
- `-like` wildcard vs `-match` regex
- `-contains` / `-in` — collection membership
- Array on left — operator filters elements

Logical, Null & Special Operators

OVERVIEW

Logical operators are –and, –or, –xor and –not (or !), and they short-circuit. PowerShell 7 added C#-style operators: null-coalescing ??, null-conditional ?., the ternary ? :, and pipeline chains && / ||. The call operator & runs a command stored in a string or script block.

NOTES

- All of `?? ?. ? : && ||` are 7.0+ (not in 5.1)
- `?.` needs braces: `${obj}?.Name`, since `?` is legal in names
- `&&` / `||` act on the previous command's `$?`
- Precedence: `-not` binds tighter than `-and` / `-or`

LOGIC, NULLS, CALLS

```
($age -ge 18) -and ($hasId)      # both must be true
-not (Test-Path $file)           # same as !(Test-Path $file)

$name = $inputName ?? 'Guest'    # default when $null
$config ??= @{}                  # assign only if $null
${user}?.Address?.City           # $null instead of error
${items}?[0]                     # safe index

$label = $n -gt 1 ? 'items' : 'item'

git pull && npm test             # test only if pull worked
npm test || Write-Warning 'fail' # runs only on failure

& 'C:\Program Files\Tool\tool.exe' --version
$sb = { param($x) $x * 2 }; & $sb 21     # 42
. .\helpers.ps1                  # load functions here

Start-Sleep 10 &                 # background job (7+)
```

CODE EXPLAINED

→ `-and / -or` — short-circuit: right side skipped when result is known

→ `??` — returns the left side unless it is `$null`

→ `${user}?.Address` — stops at the first `$null` instead of throwing

→ `? :` — expression form of if/else; returns a value

→ `&&` — runs right side only if the left command succeeded

→ `&` — call a path with spaces or a script block

LOGICAL

OP	TRUE WHEN
-and	both true
-or	either true
-xor	exactly one true
-not / !	operand false

POWERSHELL 7 OPERATORS

OP	EXAMPLE
??	$a ?? 'default'
??=	$a ??= 0
?.	${a}?.Prop
?[]	${a}?[0]
? :	$ok ? 'y' : 'n'
&& ||	cmd1 && cmd2

SPECIAL OPERATORS

OP	PURPOSE
&	call operator
.	dot-source (same scope)
$( )	subexpression
@( )	always an array
::	static member: [math]::PI
,	build an array

⚠ COMMON MISTAKES

- Writing `$user?.Name` without braces. PowerShell reads a variable called `user?`; use `${user}?.Name`
- Using `??` to replace empty strings. It only checks `$null`; `'' ?? 'x'` returns `''`
- Running a quoted path without `&`. `'C:\My Tool\x.exe'` just prints the string

✓ TIPS & BEST PRACTICES

- Use `??` to replace `if (-not $x) { $x = … }` blocks for defaults
- Use `&&` in build scripts so a failing step stops the chain
- Wrap each side of `-and` / `-or` in parentheses when it contains a command

QUICK SUMMARY

- `-and -or -xor -not` — short-circuit logic
- `??` / `??=` — default for `$null` (7+)
- `? :` / `?.` — ternary and null-conditional (7+)
- `&` / `.` — call vs dot-source

if & switch

OVERVIEW

`if` tests any expression for truthiness (non-zero, non-empty, non-null). `switch` is far more powerful than in C-like languages: it can match wildcards or regex, read a file line by line, iterate an array, and run **every** matching branch unless you `break`. Both can be assigned to a variable.

NOTES

- Falsy: `$false`, `0`, `''`, `$null`, empty array `@()`
- In `switch`, `$_` is the current value
- `switch` on an array tests each element in turn
- Use `break` to stop after the first match

BRANCHING

```
if ($free -lt 10GB) {
    Write-Warning 'Low disk'
} elseif ($free -lt 50GB) {
    'Watch it'
} else { 'Plenty' }

$size = if ($n -gt 100) { 'big' } else { 'small' }

switch ($code) {
    200             { 'OK'; break }
    { $_ -ge 500 } { 'Server error'; break }
    default         { "Code $_" }
}

switch -Wildcard ($file) { '*.log' { 'log' }  '*.txt' { 'text' } }
switch -Regex ($line) { '^ERROR (\d+)' { "err $($Matches[1])" } }
switch -File .\app.log { { $_ -like '*WARN*' } { $warn++ } }

$type = switch ($ext) { '.ps1' { 'script' } default { 'other' } }
```

CODE EXPLAINED

- → `elseif` — one word; no `else if` needed (though it works)
- → `$size = if …` — statements return values you can capture
- → `{ $_ -ge 500 }` — script block condition, true/false
- → `default` — runs when nothing else matched
- → `-Regex` — regex match; `$Matches` is available inside
- → `-File` — reads the file and switches on each line

TRUTHINESS

VALUE	IS
`$null / 0 / ''`	False
`@()`	False
`@(0)`	False (single falsy item)
`'False'`	True (non-empty)
`@(1, 2)`	True
`' '`	True

SWITCH OPTIONS

OPTION	EFFECT
`-Exact`	literal match (default)
`-Wildcard`	* ? [ ] patterns
`-Regex`	regex patterns
`-CaseSensitive`	case matters
`-File path`	iterate file lines

FLOW IN SWITCH

KEYWORD	EFFECT
`(none)`	keep testing other branches
`break`	leave the switch
`continue`	next input element
`return`	leave the function

⚠ COMMON MISTAKES

- Forgetting `break` when one value matches several branches. Every matching branch runs, unlike C#
- Writing `if ($a = 5)`. That assigns 5 and is always true; use `-eq`
- Testing a single-item array `@(0)`. It is falsy because PowerShell unwraps one element

✓ TIPS & BEST PRACTICES

- Use `switch` with script-block conditions instead of long `elseif` chains
- Use `switch -Regex -File` to parse logs in a few lines
- Prefer `if (-not (Test-Path $p))` over `if (!(Test-Path $p) -eq $true)`

QUICK SUMMARY

- `if / elseif / else` — truthy tests
- `switch` — exact, wildcard, regex, file
- `break` / `continue` — stop or skip in switch
- `$x = if …` — capture a statement's value

Loops

OVERVIEW

PowerShell has the classic loops plus `foreach` for collections. Loops are statements whose output can be **captured**: `$squares = foreach ($n in 1..5) { $n * $n }` builds an array without +=. The `foreach` *statement* is different from `ForEach-Object` in the pipeline (sheet 16).

NOTES

- `foreach` loads the whole collection first; fastest for data in memory
- `do…while` and `do…until` always run at least once
- Labels let `break outer` exit nested loops
- Looping over `$null` runs zero times (3.0+)

LOOPING PATTERNS

```
foreach ($file in Get-ChildItem *.log) {
    "$($file.Name): $($file.Length) bytes"
}

for ($i = 0; $i -lt 5; $i++) { "Line $i" }

while (-not (Test-Path .\done.flag)) { Start-Sleep 2 }   # poll
do { $answer = Read-Host 'Continue? (y/n)' } until ($answer -in 'y', 'n')

$squares = foreach ($n in 1..5) { $n * $n }   # 1 4 9 16 25

:outer foreach ($row in 1..3) {
    foreach ($col in 1..3) {
        if ($col -eq 2) { continue }      # skip this column
        if ($row -eq 3) { break outer }   # leave both loops
        "$row,$col"
    }
}
```

CODE EXPLAINED

- → `foreach ($x in …)` — iterates any collection or command output
- → `for (init; test; step)` — index-based loop
- → `while (…)` — tests before each pass
- → `do { } until (…)` — runs until the condition becomes true
- → `$squares = foreach` — collects every value emitted in the loop
- → `:outer / break outer` — label targets an outer loop

WHICH LOOP?

LOOP	BEST FOR
`foreach`	items already in memory
`ForEach-Object`	streaming pipeline
`.ForEach()`	fast method on arrays
`for`	index needed / step by 2
`while`	unknown count, test first
`do…until`	run at least once

LOOP CONTROL

KEYWORD	EFFECT
`break`	exit the loop
`continue`	next iteration
`break label`	exit labeled loop
`return`	exit function/script

COUNTING PATTERNS

PATTERN	GIVES
`1..10`	1 to 10
`10..1`	10 down to 1
`0..($a.Count-1)`	array indexes
`for (…; $i += 2)`	step by 2
`1..3 | % { }`	pipeline form

⚠ COMMON MISTAKES

- Using `break` inside `ForEach-Object`. It isn't a loop statement, so `break` exits the enclosing loop or script; use `return` to skip an item
- Building results with `$arr += $x` in big loops. It copies the array each time; capture the loop output instead
- Writing `while ($i < 5)`. `<` is reserved; use `-lt`

✓ TIPS & BEST PRACTICES

- Capture loop output directly: `$names = foreach ($u in $users) { $u.Name }`
- Use `foreach` over `ForEach-Object` for speed when the data is already in a variable
- Add `Start-Sleep` to polling loops so they don't spin the CPU

QUICK SUMMARY

- `foreach ($x in $list)` — iterate a collection
- `for` / `while` / `do…until` — classic loops
- `break` / `continue` / `:label` — control flow
- `$r = foreach …` — collect output without `+=`

Pipeline Basics

OVERVIEW

The pipeline (`|`) passes **whole objects** from one command to the next, one at a time, so you filter and sort on real properties like `CPU` or `Length` instead of parsing text. A handful of `*-Object` cmdlets do most of the work: filter, transform, pick, sort, group, and measure.

NOTES

- Objects stream: the next command starts before the first finishes
- Inside script blocks, `$_` is the current object
- **Filter left**: use a cmdlet's own `-Filter` when it has one
- Line breaks are allowed after a `|`

THE CORE PIPELINE CMDLETS

```
Get-Process | Where-Object { $_.CPU -gt 100 }      # script block form
Get-Service | Where-Object Status -eq 'Running'    # simplified form

Get-ChildItem *.log | ForEach-Object { $_.Name.ToUpper() }
Get-ChildItem | ForEach-Object Name                # member shortcut

Get-Process | Sort-Object CPU -Descending | Select-Object -First 5
Get-Process | Select-Object Name, Id, CPU
Get-Process | Select-Object -ExpandProperty Name   # plain strings

Get-ChildItem -File | Group-Object Extension |
    Sort-Object Count -Descending

Get-ChildItem -File -Recurse | Measure-Object Length -Sum -Average

(1..10).Where({ $_ % 2 -eq 0 })                    # 2 4 6 8 10
(1..3).ForEach({ $_ * 10 })                        # 10 20 30
```

CODE EXPLAINED

- → `Where-Object` — keeps objects where the test is true
- → `ForEach-Object` — runs a block for each object; output flows on
- → `-ExpandProperty` — returns the property value, not an object with one property
- → `Group-Object` — buckets by property; output has `Name`, `Count`, `Group`
- → `Measure-Object` — Count, Sum, Average, Minimum, Maximum
- → `.Where() / .ForEach()` — array methods; faster than the cmdlets on in-memory data

CORE *-OBJECT CMDLETS

CMDLET	ALIAS	JOB
`Where-Object`	? where	filter
`ForEach-Object`	% foreach	transform
`Select-Object`	select	pick / limit
`Sort-Object`	sort	order
`Group-Object`	group	bucket
`Measure-Object`	measure	stats

SELECT-OBJECT OPTIONS

OPTION	EFFECT
`-First 5 / -Last 5`	take from start / end
`-Skip 10`	skip items
`-Unique`	drop duplicates
`-Property *`	show every property
`-ExpandProperty X`	output X's value
`-Index 0,2`	pick positions

WHERE METHODS

MODE	EXAMPLE
`'First'`	.Where({$_ -gt 2}, 'First')
`'Last'`	.Where({…}, 'Last')
`'Split'`	$a, $b = $x.Where({…}, 'Split')
`'SkipUntil'`	skip until true
`count`	.Where({…}, 'First', 3)

⚠ COMMON MISTAKES

- Filtering late: `Get-ChildItem -Recurse | Where-Object Name -like '*.log'` reads everything; `-Filter *.log` is far faster
- Using `Select-Object Name` when you need strings. You get objects with a Name property; use `-ExpandProperty`
- Sorting text that holds numbers. `'10','9' | Sort-Object` → 10, 9; sort on `{ [int]$_ }`

✓ TIPS & BEST PRACTICES

- Pipe to `Get-Member` mid-pipeline to see what the next command receives
- Sort by several keys: `Sort-Object Department, @{e='Salary'; Descending=$true}`
- Use `Select-Object -First 1` to stop the upstream command early

QUICK SUMMARY

- `Where-Object` — filter by property
- `ForEach-Object` — act on each object
- `Select-Object` / `Sort-Object` — pick and order
- `Group-Object` / `Measure-Object` — bucket and summarize

Custom Objects & Calculated Properties

OVERVIEW

Build your own output objects with `[PSCustomObject]@{ }`: they sort, filter, export to CSV/JSON and format just like built-in objects. **Calculated properties** add or reshape properties on the fly inside `Select-Object`, `Sort-Object` or `Format-Table` using a small hashtable with a name and an expression.

NOTES

- `[PSCustomObject]` keeps keys in the order you wrote them
- Keys: `Name` (or `n`, `Label`, `l`) and `Expression` (`e`)
- Return objects from functions, not formatted text
- `Select-Object` outputs new objects (a copy)

BUILDING AND RESHAPING OBJECTS

```
$report = [PSCustomObject]@{
    Server = 'web01'
    CPU    = 37.5
    Online = $true
}                                  # $report.Server → web01

Get-ChildItem -File | Select-Object Name,
    @{ Name = 'MB'; Expression = { [math]::Round($_.Length / 1MB, 2) } },
    @{ n = 'Age'; e = { ((Get-Date) - $_.LastWriteTime).Days } }

Get-Process | Select-Object * -ExcludeProperty *Memory*

$report | Add-Member -NotePropertyName Owner -NotePropertyValue 'ops'
$report | Add-Member -MemberType ScriptProperty -Name Busy `
    -Value { $this.CPU -gt 80 }

Compare-Object (Get-Content a.txt) (Get-Content b.txt)
Get-Process | Tee-Object -Variable procs | Sort-Object CPU
```

CODE EXPLAINED

- → `[PSCustomObject]@{}` — ordered, typed object; fastest way to build output
- → `@{Name=…; Expression={…}}` — calculated property; `$_` is the input object
- → `n / e` — short keys for Name and Expression
- → `-ExcludeProperty` — works with `*` to drop properties
- → `ScriptProperty` — computed every time it's read; `$this` is the object
- → `Compare-Object` — `<=` only in left, `=>` only in right

CALCULATED PROPERTY KEYS

KEY	USE
`Name / n / Label / l`	new property name
`Expression / e`	script block or name
`FormatString`	Format-Table only: 'N2'
`Width / Alignment`	Format-Table only
`Descending`	Sort-Object only

ADD-MEMBER TYPES

MEMBERTYPE	ADDS
`NoteProperty`	fixed value
`ScriptProperty`	computed value
`ScriptMethod`	callable method
`AliasProperty`	alternate name
`-PassThru`	return the object

COMPARE-OBJECT

PARAMETER	EFFECT
`-Property Name`	compare one property
`-IncludeEqual`	show == too
`-ExcludeDifferent`	only matches
`-PassThru`	return the originals

⚠ COMMON MISTAKES

- Using `New-Object PSObject -Property @{}`. Property order is lost; `[PSCustomObject]@{}` keeps it and is faster
- Building strings like `"$name - $size"` as function output. Callers can't sort or export text; return objects
- Forgetting `-PassThru` on `Add-Member` mid-pipeline, so nothing flows on

✓ TIPS & BEST PRACTICES

- Build arrays of objects in a loop: `$rows = foreach ($s in $servers) { [PSCustomObject]@{…} }`
- Use `Select-Object -Property *,@{n='X';e={…}}` to keep all properties and add one
- Use `$obj.PSObject.Properties` to loop over an object's property names and values

QUICK SUMMARY

- `[PSCustomObject]@{}` — create an ordered object
- `@{n='X'; e={…}}` — calculated property
- `Add-Member` — add properties after creation
- `Compare-Object` — diff two sets

Formatting Output

OVERVIEW

`Format-*` cmdlets control how objects *look* on screen. They convert objects into formatting instructions, so they must come **last** in a pipeline (the **format-right rule**). If you need to save or process the data, use `Select-Object` and `Export-Csv` instead.

NOTES

- Objects with 5+ properties default to a list view
- `Format-Table` truncates long values with `…` unless `-Wrap`
- `$FormatEnumerationLimit` (default 4) cuts off arrays in output
- `Out-GridView` is Windows-only (7: GraphicalTools module)

SHAPING SCREEN OUTPUT

```
Get-Process | Format-Table Name, Id, CPU -AutoSize
Get-Service | Format-Table -GroupBy Status -Property Name
Get-ChildItem | Format-Table Name,
    @{ n = 'KB'; e = { $_.Length / 1KB }; FormatString = 'N1' }

Get-Process pwsh | Format-List *        # every property
Get-Process pwsh | Format-List Name, Path, StartTime
Get-ChildItem | Format-Wide Name -Column 4

$text = Get-Process | Format-Table | Out-String -Width 200
Get-Service | Out-GridView -PassThru | Restart-Service

Get-Process | Select-Object Name, CPU |
    ConvertTo-Html -Title 'Processes' | Out-File procs.html

$FormatEnumerationLimit = -1            # show full arrays
```

CODE EXPLAINED

- → `-AutoSize` — fits columns to content (waits for all input)
- → `-GroupBy` — a sub-table per group; sort by that property first
- → `FormatString` — number format inside a calculated column
- → `Format-List *` — best way to see all properties and values
- → `Out-String` — render formatted output into a string
- → `-PassThru` — Out-GridView returns the rows you select

FORMAT CMDLETS

CMDLET	ALIAS	LAYOUT
`Format-Table`	ft	columns
`Format-List`	fl	one property per line
`Format-Wide`	fw	one property, many columns
`Format-Custom`	fc	tree view

FORMAT-TABLE OPTIONS

OPTION	EFFECT
`-AutoSize`	fit widths
`-Wrap`	wrap instead of …
`-HideTableHeaders`	no header row
`-GroupBy X`	group rows
`-View name`	predefined view

RIGHT VS WRONG

PIPELINE	RESULT
`… | Select | Export-Csv`	✔ real data
`… | ft | Export-Csv`	✖ junk rows
`… | Sort | ft`	✔ sorted table
`… | ft | Sort`	✖ error / junk
`… | ft | Out-File`	✔ text file

⚠ COMMON MISTAKES

- Piping `Format-Table` into `Export-Csv` or `Where-Object`. You get formatting objects, not data; format last
- Returning `Format-Table` output from a function. Callers can't use it; return objects and let them format
- Expecting `Out-GridView` on Linux/macOS. It needs the GraphicalTools module there

✓ TIPS & BEST PRACTICES

- Use `Format-List *` to discover property names quickly
- Use `ft -AutoSize -Wrap` for long paths or messages
- Use `$PSStyle.Formatting` (7.2+) to change table header colors

QUICK SUMMARY

- `Format-Table` / `ft` — columns
- `Format-List` / `fl` — property list; `fl *` shows all
- Format right — always the last command
- `Out-String` / `ConvertTo-Html` — formatted text or HTML

Errors & ErrorAction

OVERVIEW

PowerShell has two kinds of errors. A **non-terminating** error (most cmdlet errors) is reported and the command *keeps going* with the next item. A **terminating** error stops the statement and can be caught with `try/catch`. `-ErrorAction Stop` turns a non-terminating error into a terminating one.

ERROR BEHAVIOR

```
Get-Item a.txt, nope.txt, b.txt   # error for nope.txt; a, b still return

Get-Item nope.txt -ErrorAction SilentlyContinue   # quiet, still in $Error
Get-Item nope.txt -ErrorAction Ignore             # quiet, not recorded
Get-Item nope.txt -ErrorAction Stop               # terminating → catchable

Get-ChildItem C:\Windows -Recurse -ErrorAction SilentlyContinue `
    -ErrorVariable errs
$errs.Count                       # how many were skipped

$Error[0]                         # last error
$Error[0].InvocationInfo.ScriptLineNumber
Get-Error                         # detailed view (7+)

Write-Error 'Config missing' -Category ObjectNotFound
$ErrorActionPreference = 'Stop'   # whole script: all errors terminate
```

NOTES

- `$Error` holds recent errors, newest first: `$Error[0]`
- `-ErrorAction` is a common parameter on every cmdlet
- `Ignore` hides the error and skips `$Error`; `SilentlyContinue` still logs it
- Native exe failures set `$LASTEXITCODE`, not errors

CODE EXPLAINED

- → `a.txt, nope.txt, b.txt` — non-terminating: one bad item doesn't stop the rest
- → `SilentlyContinue` — hide the message but keep a record
- → `-ErrorVariable errs` — collect this command's errors; `+errs` appends
- → `InvocationInfo` — where the error happened: line, command
- → `Get-Error` — expands the full error record and inner exceptions
- → `Write-Error` — emit a non-terminating error from your code

-ERRORACTION VALUES

VALUE	EFFECT
`Continue`	show, keep going (default)
`SilentlyContinue`	hide, keep going
`Ignore`	hide, don't record
`Stop`	make terminating
`Inquire`	ask the user
`Break`	enter debugger (7+)

ERROR KINDS

SOURCE	KIND
`most cmdlet errors`	non-terminating
`throw`	terminating
`-ErrorAction Stop`	terminating
`parse error`	script won't start
`.NET exception`	terminating
`exe exit code`	neither (check it)

ERRORRECORD PARTS

PROPERTY	HOLDS
`Exception`	.NET exception + Message
`CategoryInfo`	category + target
`FullyQualifiedErrorId`	stable ID to match on
`InvocationInfo`	line, command
`TargetObject`	the failing item
`ScriptStackTrace`	call stack

⚠ COMMON MISTAKES

- Wrapping a cmdlet in `try/catch` without `-ErrorAction Stop`. Non-terminating errors skip the `catch` entirely
- Using `SilentlyContinue` everywhere to hide noise. Real failures disappear; collect them with `-ErrorVariable` instead
- Assuming `git` or `robocopy` failures throw. Check `$LASTEXITCODE` after native commands

✓ TIPS & BEST PRACTICES

- Set `$ErrorActionPreference = 'Stop'` at the top of automation scripts so nothing fails silently
- Use `$Error[0] | Format-List * -Force` in 5.1 where `Get-Error` doesn't exist
- Use `-ErrorVariable` on long recursive scans to report skipped items at the end

QUICK SUMMARY

- Non-terminating — reported, command continues
- `-ErrorAction Stop` — make it terminating / catchable
- `$Error[0]` / `Get-Error` — inspect the last error
- `-ErrorVariable` — capture errors for one command

try / catch / finally & throw

OVERVIEW

`try` runs code, `catch` handles **terminating** errors, and `finally` always runs (cleanup). Catch specific exception types first, then a general `catch`. Inside `catch`, `$_` is the ErrorRecord. Use `throw` to raise your own terminating error, or a bare `throw` to rethrow.

NOTES

- Only terminating errors reach `catch`: add `-ErrorAction Stop`
- First matching `catch` wins; put specific types first
- `finally` runs even after `return`, `break` or Ctrl+C
- Find a type to catch with `$_.Exception.GetType().FullName`

STRUCTURED ERROR HANDLING

```powershell
try {
    $data = Get-Content .\config.json -ErrorAction Stop | ConvertFrom-Json
    $stream = [IO.File]::OpenRead($data.LogPath)
}
catch [System.Management.Automation.ItemNotFoundException] {
    Write-Warning "No config file: $($_.Exception.Message)"
}
catch [System.ArgumentException], [System.FormatException] { 'Bad config' }
catch {
    Write-Error "Unexpected: $($_.Exception.GetType().FullName)"
    throw                       # rethrow the same error
}
finally {
    if ($stream) { $stream.Dispose() }   # always runs
}

if (-not $Path) { throw 'Path is required' }
throw [System.IO.FileNotFoundException]::new('Missing', $Path)
```

CODE EXPLAINED

- → `-ErrorAction Stop` — needed so the cmdlet error is catchable
- → `catch [Type]` — matches that type or any subclass
- → `[A], [B]` — one block can catch several types
- → `$_.Exception` — the .NET exception: `.Message`, `.InnerException`
- → `throw` — bare in `catch` = rethrow, preserving details
- → `finally` — close files, connections, restore settings

COMMON EXCEPTION TYPES

TYPE	RAISED WHEN
`ItemNotFoundException`	path doesn't exist
`UnauthorizedAccessException`	access denied
`IOException`	file in use / disk
`ArgumentException`	bad argument
`WebException / HttpRequestException`	web call failed (5.1 / 7)
`RuntimeException`	plain `throw 'msg'`

RAISING & TRAPPING

FORM	USE
`throw 'msg'`	quick terminating error
`throw [Type]::new('m')`	typed exception
`throw`	rethrow inside catch
`Write-Error -EA Stop`	terminating ErrorRecord
`ThrowTerminatingError()`	via `$PSCmdlet` in advanced functions
`trap { …; continue }`	older style: handle, resume

⚠ COMMON MISTAKES

- Catching but not rethrowing in a library function. The caller thinks it worked; log and `throw`
- Using `throw $_` instead of `throw`. It works but resets details in some cases; a bare `throw` keeps the original
- Putting cleanup after the try block instead of in `finally`. It's skipped when an error escapes

✓ TIPS & BEST PRACTICES

- Log `$_.ScriptStackTrace` in catch blocks to see where it failed
- Run the failing line once, then check `$Error[0].Exception.GetType()` to learn which type to catch
- Keep `try` blocks small so you know which line failed

QUICK SUMMARY

- `try / catch / finally` — handle terminating errors, always clean up
- `catch [Type]` — specific first, general last
- `$_.Exception.Message` — the error text
- `throw` — raise, or rethrow when bare

Functions & Script Blocks

OVERVIEW

A function is a named script block. Declare parameters in a `param()` block and call the function like a cmdlet: **space-separated, no parentheses or commas**. Everything the function emits is its output, not just what follows `return`. A **script block** `{ }` is code stored as a value that you run later with &.

NOTES

- Call as `Add-Num 1 2`, never `Add-Num(1, 2)`
- `return $x` = output `$x`, then exit
- Name functions `Verb-Noun` with approved verbs
- Splat with `@name`, not `$name`

DEFINING AND CALLING

```
function Get-Greeting {
    param(
        [string]$Name = 'World',
        [switch]$Shout
    )
    $msg = "Hello, $Name!"
    if ($Shout) { return $msg.ToUpper() }
    $msg
}
Get-Greeting                      # Hello, World!
Get-Greeting -Name Ada -Shout     # HELLO, ADA!

$p = @{ Path = 'C:\Logs'; Filter = '*.log'; Recurse = $true }
Get-ChildItem @p                  # hashtable splatting

$double = { param($x) $x * 2 }
& $double 21                      # 42
Invoke-Command -ScriptBlock $double -ArgumentList 5   # 10
```

CODE EXPLAINED

- → `param()` — typed parameters with defaults
- → `[switch]` — on/off flag; `$Shout` is True only when passed
- → `return` — exits early; any earlier output is still returned
- → `@p` — spreads hashtable keys as named parameters
- → `{ param($x) … }` — script block with its own parameters
- → `& $double 21` — invoke a script block like a command

CALLING FUNCTIONS

FORM	RESULT
`Add-Num 1 2`	✔ positional
`Add-Num -A 1 -B 2`	✔ named
`Add-Num(1, 2)`	✖ one array arg
`Add-Num 1, 2`	✖ one array arg
`$r = Add-Num 1 2`	✔ capture output
`(Add-Num 1 2) * 3`	✔ use in expression

DECLARATION STYLES

STYLE	EXAMPLE
`param block`	function F { param($a) }
`inline`	function F($a, $b) { }
`filter`	filter Even { if ($_%2 -eq 0) {$_} }
`no params`	$args[0], $args[1]

SCRIPT BLOCK USES

WHERE	EXAMPLE
`Where-Object`	{ $_.CPU -gt 5 }
`& / Invoke`	& $sb
`Invoke-Command`	-ScriptBlock { }
`Start-Job`	-ScriptBlock { }
`Calculated prop`	e = { $_.Len }
`Variable`	$sb = { Get-Date }

⚠ COMMON MISTAKES

- Calling with parentheses and commas: `Add-Num(1, 2)` passes one array to the first parameter; use `Add-Num 1 2`
- Leaving stray output: a bare `$list.Add(1)` or `New-Item` inside the function becomes part of the return value
- Splatting with `$p` instead of `@p`. `$p` passes the whole hashtable as one argument

✓ TIPS & BEST PRACTICES

- Splat long parameter lists to keep lines short and readable
- Type every parameter (`[string]`, `[int]`) so bad input fails early
- Use `$PSBoundParameters` to see which parameters the caller passed

QUICK SUMMARY

- `function Verb-Noun { param() }` — define
- `Name arg1 arg2` — call with spaces, no parens
- Output = everything emitted, `return` just exits
- `@params` — splatting; `& {}` — run a script block

Advanced Functions & Parameter Validation

OVERVIEW

Adding `[CmdletBinding()]` turns a function into an **advanced function** that behaves like a compiled cmdlet: it gets common parameters (`-Verbose`, `-ErrorAction`…), and its parameters can be mandatory, positional, grouped into sets, and **validated** before your code runs. Validation attributes replace most hand-written input checks.

NOTES

- Any `[Parameter()]` attribute also makes it advanced
- `Mandatory` prompts if the caller omits it
- Validation failures throw before the body runs
- `$PSCmdlet.ParameterSetName` tells you which set was used

AN ADVANCED FUNCTION

```
function New-AppUser {
    [CmdletBinding(DefaultParameterSetName = 'ByName')]
    param(
        [Parameter(Mandatory, Position = 0, ParameterSetName = 'ByName')]
        [ValidateNotNullOrEmpty()]
        [string]$UserName,

        [Parameter(Mandatory, ParameterSetName = 'ById')]
        [ValidatePattern('^\d{6}$')] [string]$EmployeeId,

        [ValidateSet('Dev', 'Test', 'Prod')] [string]$Environment = 'Dev',

        [ValidateScript({ Test-Path $_ -PathType Container })]
        [string]$HomeDir = $HOME
    )
    Write-Verbose "Set: $($PSCmdlet.ParameterSetName), env: $Environment"
}
```

CODE EXPLAINED

- → `DefaultParameterSetName` — set used when the input is ambiguous
- → `Mandatory` — required; same as `Mandatory = $true`
- → `Position = 0` — allows `New-AppUser bob` without `-UserName`
- → `ParameterSetName` — UserName and EmployeeId can't be used together
- → `[ValidateSet()]` — also gives tab completion of the values
- → `Write-Verbose` — shown only when called with `-Verbose`

VALIDATION ATTRIBUTES

ATTRIBUTE	CHECKS
`[ValidateSet('a','b')]`	one of a list
`[ValidateRange(1,10)]`	numeric range
`[ValidatePattern('re')]`	regex match
`[ValidateScript({…})]`	custom test → $true
`[ValidateNotNullOrEmpty()]`	has a value

[PARAMETER()] ARGUMENTS

ARGUMENT	EFFECT
`Mandatory`	required
`Position = 0`	positional slot
`ParameterSetName`	belongs to a set
`HelpMessage`	text at the prompt
`ValueFromPipeline`	see sheet 23

COMMON PARAMETERS

PARAMETER	EFFECT
`-Verbose / -Debug`	show those streams
`-ErrorAction`	error behavior
`-OutVariable`	also save output
`-ErrorVariable`	save errors
`-WhatIf / -Confirm`	with ShouldProcess

⚠ COMMON MISTAKES

- Testing `if ($Days)` to see if a parameter was passed. Defaults and 0 fool it; check `$PSBoundParameters` keys
- Giving a `Mandatory` parameter a default value. The default is ignored; the user is prompted anyway
- Writing manual `if (-not ($x -in …)) { throw }` checks that a `[ValidateSet()]` does better

✓ TIPS & BEST PRACTICES

- Add `[Alias('CN')]` to support familiar alternate parameter names
- Use `[ValidateScript({ … })]` with a `throw 'message'` inside to give a clear error
- Use `[ArgumentCompleter()]` for dynamic tab completion (e.g. server names)

QUICK SUMMARY

- `[CmdletBinding()]` — cmdlet-like function + common params
- `[Parameter(Mandatory, Position=0)]` — required, positional
- `ParameterSetName` — mutually exclusive groups
- `[Validate*()]` — check input before the body runs

Pipeline Input & ShouldProcess

OVERVIEW

To accept pipeline input, mark a parameter `ValueFromPipeline` (whole objects) or `ValueFromPipelineByPropertyName` (match a property by name), then put per-item work in a `process` block. Functions that change things should support `–WhatIf` and `–Confirm` by declaring `SupportsShouldProcess` and checking `$PSCmdlet.ShouldProcess()`.

NOTES

- Without a `process` block, the body runs once, in `end`
- `begin` runs once before input, `end` once after
- `clean` (7.3+) always runs, even on Ctrl+C or errors
- ByPropertyName also matches parameter `[Alias()]` names

PIPELINE-AWARE, SAFE FUNCTION

```
function Remove-OldLog {
    [CmdletBinding(SupportsShouldProcess, ConfirmImpact = 'Medium')]
    param([Parameter(Mandatory, ValueFromPipeline)] [IO.FileInfo[]]$File,
          [int]$Days = 30)
    begin   { $cutoff = (Get-Date).AddDays(-$Days); $n = 0 }
    process {
        foreach ($f in $File) {
            if ($f.LastWriteTime -lt $cutoff -and
                $PSCmdlet.ShouldProcess($f.FullName, 'Delete')) {
                Remove-Item $f.FullName; $n++
            }
        }
    }
    end { Write-Verbose "Removed $n file(s)" }
}
Get-ChildItem C:\Logs -File | Remove-OldLog -Days 7 -WhatIf
```

CODE EXPLAINED

- → `SupportsShouldProcess` — adds `-WhatIf` and `-Confirm` parameters
- → `ConfirmImpact` — prompts automatically when ≥ `$ConfirmPreference` (High)
- → `ValueFromPipeline` — each piped FileInfo binds to `$File`
- → `foreach ($f in $File)` — also handles `-File $a, $b` passed directly
- → `ShouldProcess(target, action)` — returns False under `-WhatIf` and prints "What if: …"

FUNCTION BLOCKS

BLOCK	RUNS
`begin`	once, before input
`process`	once per input item
`end`	once, after input
`clean`	always, last (7.3+)
`(no blocks)`	same as end

BINDING MODES

MODE	PIPED INPUT
`ValueFromPipeline`	the object itself
`…ByPropertyName`	obj.Name → -Name
`both`	try value, then property
`$input`	simple functions, end block
`filter keyword`	body = process block

SHOULDPROCESS PIECES

PIECE	PURPOSE
`-WhatIf`	describe, don't act
`-Confirm`	ask per item
`ConfirmImpact`	Low / Medium / High
`ShouldContinue()`	extra yes/no prompt
`[OutputType()]`	documents output type

⚠ COMMON MISTAKES

- Putting the work outside `process`. Only the last piped item is handled, because the body runs once in `end`
- Calling `Remove-Item` before checking `ShouldProcess`. `-WhatIf` then deletes for real
- Typing a pipeline parameter as `[string]` when callers pass arrays directly; use `[string[]]` plus `foreach`

✓ TIPS & BEST PRACTICES

- Test new destructive functions with `-WhatIf` first, every time
- Use ByPropertyName so output from `Get-ADUser`, `Import-Csv` etc. binds by column name
- Use `[OutputType([pscustomobject])]` so tooling knows what you return

QUICK SUMMARY

- `ValueFromPipeline` / `…ByPropertyName` — accept input
- `begin / process / end` — setup, per item, wrap up
- `SupportsShouldProcess` — add `-WhatIf` / `-Confirm`
- `$PSCmdlet.ShouldProcess()` — guard every change

Arrays & Lists

OVERVIEW

Arrays are fixed-size, so += silently creates a new copy each time. That's fine for a few items but slow for thousands; use `[System.Collections.Generic.List[object]]` or capture loop output instead. PowerShell **unrolls** arrays in the pipeline and returns a single item as a scalar, so wrap results in `@( )` when you need an array.

NOTES

- Indexes start at 0; `-1` is the last item
- Out-of-range index returns `$null`, no error
- A command returning one item gives a scalar, not an array
- `,` builds arrays: `1, 2, 3`; `,$x` = one-item array

ARRAYS AND LISTS

```
$a = 1, 2, 3                    # array
$empty = @()
$one = ,'only'                  # 1-item array
$a[0]; $a[-1]                   # 1; 3
$a[0..1]                        # 1 2
$a[-2..-1]                      # 2 3
$a.Count                        # 3
$a += 4                         # copy of array + 4
3 -in $a                        # True

$files = @(Get-ChildItem *.cfg) # always an array
$files.Count                    # 0, 1 or more, never $null

$list = [System.Collections.Generic.List[object]]::new()
$list.Add('a'); $list.Add('b')  # no copying, no output
$list.Remove('a')               # True
$list.Insert(0, 'z'); $list     # z b
```

CODE EXPLAINED

- → `1, 2, 3` — comma operator creates `Object[]`
- → `$a[-2..-1]` — ranges as indexes give slices
- → `+=` — allocates a new array each time
- → `@( … )` — array subexpression; guarantees an array result
- → `List[object]` — growable; `Add`, `Remove`, `Insert`, `Contains`
- → `$list.Add()` — List's Add returns nothing (ArrayList returns an index)

ARRAY OPERATIONS

EXPRESSION	RESULT
`$a.Count / .Length`	number of items
`$a -contains 2`	membership
`$a -join ','`	1,2,3
`$a | Sort-Object -Desc`	sorted copy
`[array]::Reverse($a)`	reverse in place
`$a.IndexOf(2)`	1

LIST[T] METHODS

METHOD	EFFECT
`.Add(x)`	append
`.AddRange(@(…))`	append many
`.Remove(x)`	first match, returns bool
`.RemoveAt(i)`	by index
`.Contains(x)`	membership
`.ToArray()`	back to array

TYPED ARRAYS

FORM	HOLDS
`[int[]]$n = 1,2`	ints only
`[string[]]`	strings
`List[int]`	typed list
`[object[]]`	anything (default)
`[array]$x`	force array

⚠ COMMON MISTAKES

- Using `+=` to build a 10,000-item array. It's O(n²); capture loop output or use a `List`
- Checking `$result.Count` when a command returned one object. Always wrap: `@(Get-Thing).Count`
- Using `ArrayList.Add()` without discarding. It outputs the new index into your results

✓ TIPS & BEST PRACTICES

- Prefer `$x = foreach (…) { … }` to build arrays; it's fast and needs no `+=`
- Use `$a.Where({ … }, 'First')` to stop at the first match
- Return an array intact from a function with `,$arr` or `Write-Output -NoEnumerate`

QUICK SUMMARY

- `1,2,3` / `@()` — create arrays
- `$a[-1]`, `$a[0..2]` — index and slice
- `+=` copies — use `List[object]` for many adds
- `@( … )` — force an array result

Hashtables & Dictionaries

OVERVIEW

A hashtable stores **key/value pairs** with fast lookup by key. Keys are case-insensitive strings by default and order isn't kept; use `[ordered]@{}` when order matters. Hashtables are everywhere in PowerShell: splatting, calculated properties, `[PSCustomObject]` literals, config data and lookup tables.

NOTES

- Access with dot or index: `$h.Name` or `$h['Name']`
- Missing keys return `$null`, no error
- You can't add keys while looping over `.Keys`
- For typed keys/values use `Dictionary[K,V]`

WORKING WITH HASHTABLES

```
$ages = @{ Ada = 36; Linus = 54 }
$ages.Ada                          # 36
$ages['Linus']                     # 54
$ages.Grace = 85                   # add or update
$ages.Add('Alan', 41)              # error if key exists
$ages.ContainsKey('Ada')           # True
$ages.Remove('Alan')

foreach ($kv in $ages.GetEnumerator()) { "$($kv.Key) is $($kv.Value)" }
$ages.GetEnumerator() | Sort-Object Value -Descending

$cfg = [ordered]@{ Name = 'web'; Port = 443; Tls = $true }
$cfg.Keys                          # Name Port Tls (in order)

$counts = [System.Collections.Generic.Dictionary[string,int]]::new()
$counts['err'] = 1
$counts.ContainsKey('ERR')         # False (case-sensitive)
$seen = [System.Collections.Generic.HashSet[string]]::new()
$seen.Add('a'); $seen.Add('a')     # True; False (duplicate)
```

CODE EXPLAINED

- → `$ages.Grace = 85` — assignment adds a new key or replaces a value
- → `.Add(k, v)` — strict add; throws on duplicates
- → `.GetEnumerator()` — needed to pipe or sort key/value pairs
- → `[ordered]` — keeps insertion order (OrderedDictionary)
- → `Dictionary[string,int]` — typed; keys are case-sensitive by default
- → `HashSet[string]` — unique values; `Add` returns False on duplicates

HASHTABLE MEMBERS

MEMBER	USE
`.Keys / .Values`	collections
`.Count`	pair count
`.ContainsKey(k)`	key exists?
`.ContainsValue(v)`	value exists?
`.Remove(k)`	delete key
`.Clone()`	shallow copy

COMMON PATTERNS

PATTERN	EXAMPLE
`Lookup table`	$codes[$status]
`Count items`	$c[$w]++
`Group to map`	Group-Object X -AsHashTable
`Merge`	$a + $b (dupes error)
`To object`	[PSCustomObject]$h
`From JSON`	ConvertFrom-Json -AsHashtable

WHICH COLLECTION?

TYPE	WHEN
`@{}`	general key/value
`[ordered]@{}`	order matters
`Dictionary[K,V]`	typed, many items
`HashSet[T]`	unique set

⚠ COMMON MISTAKES

- Piping a hashtable and expecting one item per key. It goes through as one object; use `.GetEnumerator()`
- Modifying `$h` inside `foreach ($k in $h.Keys)`. The collection changed error; loop over `@($h.Keys)` instead
- Expecting `@{}` to keep key order in output. Use `[ordered]@{}`

✓ TIPS & BEST PRACTICES

- Use a hashtable lookup instead of a long `switch` for code-to-name mapping
- Use `Group-Object Id -AsHashTable -AsString` to index records by key
- Use `$h.Keys | Sort-Object` for stable, readable output

QUICK SUMMARY

- `@{ k = v }` — create; `$h.k` / `$h['k']` — read
- `[ordered]@{}` — keep insertion order
- `.ContainsKey()` / `.Remove()` / `.GetEnumerator()`
- `Dictionary[K,V]` / `HashSet[T]` — typed .NET collections

Working with .NET

OVERVIEW

PowerShell runs on .NET, so the entire class library is available directly. Call **static** members with `[Type]::Member`, create objects with `[Type]::new()`, and call instance methods with a dot. When no cmdlet does what you need, there is almost always a .NET class that does.

NOTES

- Method calls need parentheses: `$s.ToUpper()`
- Type a method name without `()` to see its overloads
- `::new()` (5.0+) is faster than `New-Object`
- `using namespace` must be the first lines of a script

CALLING .NET FROM POWERSHELL

```
[math]::Sqrt(144)                       # 12
[math]::PI                              # 3.14159265358979
[int]::MaxValue                         # 2147483647
[guid]::NewGuid().Guid                  # 'a3f1…'
[IO.Path]::GetExtension('report.pdf')   # .pdf

$sb = [System.Text.StringBuilder]::new()
[void]$sb.Append('Hello, ').Append('World')
$sb.ToString()                          # Hello, World

using namespace System.Collections.Generic   # top of file only
$q = [Queue[string]]::new(); $q.Enqueue('job1')

[math] | Get-Member -Static -MemberType Method
[IO.File]::ReadAllText                  # shows overloads
Add-Type -AssemblyName System.Web
[System.Web.HttpUtility]::UrlEncode('a b&c')   # a+b%26c
```

CODE EXPLAINED

- → `[math]::Sqrt()` — static method: no object needed
- → `[int]::MaxValue` — static property
- → `::new()` — calls the constructor; arguments go in the parens
- → `[void]` — discards the return value of chained calls
- → `using namespace` — lets you write `[Queue[string]]` without the full name
- → `Add-Type -AssemblyName` — loads a .NET assembly that isn't loaded yet

USEFUL STATIC CLASSES

CLASS	EXAMPLE
`[math]`	Round, Floor, Max, Abs
`[IO.Path]`	Combine, GetFileName
`[IO.File]`	ReadAllLines, Exists
`[Environment]`	GetFolderPath('Desktop')
`[Convert]`	ToBase64String, ToInt32('ff',16)
`[Net.Dns]`	GetHostAddresses('x')

CREATING OBJECTS

FORM	NOTES
`[T]::new(a, b)`	fast, 5.0+
`New-Object T -Arg a,b`	works in 2.0+
`[T]@{ Prop = 1 }`	cast hashtable → props
`[T]'text'`	cast/parse
`New-Object -ComObject X`	COM (Windows)

EXPLORING TYPES

COMMAND	SHOWS
`$x | Get-Member`	instance members
`[T] | gm -Static`	static members
`[T].GetConstructors()`	constructors
`[T]::new`	constructor overloads
`$x.GetType().FullName`	exact type

⚠ COMMON MISTAKES

- Calling a method without parentheses. `$s.ToUpper` returns the method info, not the result
- Calling methods with spaces like a cmdlet: `$s.Replace 'a' 'b'`. Methods need `('a', 'b')`
- Putting `using namespace` below other code. It must come before any other statement

✓ TIPS & BEST PRACTICES

- Use `[T]::new` (no parens) to read constructor overloads quickly
- Prefer `StringBuilder` or `-join` when building large strings in a loop
- Use `Add-Type -TypeDefinition @' … '@` to compile small C# helpers inline

QUICK SUMMARY

- `[Type]::Member` — static method or property
- `[Type]::new()` — create an instance
- `$obj.Method()` — instance call, always with parens
- `Get-Member -Static` — discover what a type offers

Classes & Enums

OVERVIEW

PowerShell 5.0+ supports **classes** with typed properties, constructors, methods, static members and inheritance, plus **enums** for named constants. Classes are stricter than functions: methods must declare a return type and use `return`, because nothing is output implicitly. Use them for data models, DSC resources, and validation-heavy code.

NOTES

- Inside methods, use `$this.Prop` for members
- Methods return only via `return`; `[void]` for none
- Load a class before using it (define it above, or `using module`)
- Changing a class needs a new session to reload

CLASS, INHERITANCE, ENUM

```
enum Env { Dev; Test; Prod }
class Server {
    [string]$Name
    [Env]$Env = [Env]::Dev
    [int]$Port = 443
    hidden [datetime]$Created = (Get-Date)
    static [int]$Count = 0

    Server([string]$name) { $this.Name = $name; [Server]::Count++ }
    [string] Url() { return "https://$($this.Name):$($this.Port)" }
}
class WebServer : Server {
    WebServer([string]$name) : base($name) { $this.Port = 8443 }
}
$s = [WebServer]::new('web01'); $s.Url()    # https://web01:8443
[Server]::Count                             # 1
[Flags()] enum Perm { Read = 1; Write = 2; Exec = 4 }
[Perm]'Read, Write'                         # Read, Write (3)
```

CODE EXPLAINED

- → `[Env]$Env` — property typed with an enum; only valid names accepted
- → `static` — shared by the class: `[Server]::Count`
- → `Server([string]$name)` — constructor: same name as the class
- → `: base($name)` — calls the parent constructor
- → `[Flags()]` — values combine with commas or `-bor`

CLASS MEMBERS

SYNTAX	MEANING
`[type]$Prop`	typed property
`[type]$P = val`	default value
`[type] M() { }`	method + return type
`[void] M() { }`	no return
`static / hidden`	shared / not listed

ENUM USAGE

EXPRESSION	RESULT
`[Env]::Prod`	Prod
`[Env]'test'`	Test (case-insensitive)
`[int][Env]::Prod`	2
`[enum]::GetNames([Env])`	Dev Test Prod

CLASS VS PSCUSTOMOBJECT

NEED	USE
`Quick output record`	[PSCustomObject]
`Enforced types`	class
`Methods + logic`	class
`Inheritance`	class

⚠ COMMON MISTAKES

- Writing output in a method without `return`. Methods emit nothing implicitly; the result is empty
- Using a property without `$this.` inside a method. `$Name` is an undefined local variable there
- Using `Import-Module` to get a module's classes. Classes need `using module ModuleName` at the top

✓ TIPS & BEST PRACTICES

- Override `ToString()` so objects print nicely in strings and lists
- Use enums for parameters too: `param([Env]$Target)` gives validation and tab completion
- Keep classes in a module and load with `using module` for reuse

QUICK SUMMARY

- `class Name { [type]$Prop; Name() {} }` — define
- `[type] Method() { return … }` — explicit returns
- `class B : A { B() : base() {} }` — inherit
- `enum` / `[Flags()] enum` — named constants

File System & Items

OVERVIEW

File and folder commands use the generic ***-Item** cmdlets, which also work on other **providers** like the registry, environment and certificates. Build paths with `Join-Path` rather than string concatenation so they work on every OS, and use `-LiteralPath` when names contain `[ ]`.

NOTES

- Both `/` and `\` work as separators on Windows
- `-Path` supports wildcards; `-LiteralPath` doesn't
- `-Force` shows hidden files and creates parent folders
- Files are `FileInfo` objects; folders `DirectoryInfo`

FINDING AND MANAGING FILES

```
Get-ChildItem C:\Logs -Filter *.log -Recurse -File
Get-ChildItem -Directory -Depth 1          # folders, 2 levels
Get-ChildItem *.ps1, *.psm1                # several patterns

Test-Path .\out                            # True / False
Test-Path $p -PathType Leaf                # file only
New-Item .\out\reports -ItemType Directory -Force

Copy-Item .\src -Destination .\backup -Recurse
Rename-Item .\old.log -NewName new.log
Remove-Item .\temp -Recurse -Force

$cfg = Join-Path $PSScriptRoot 'config' 'app.json'   # 7+: many parts
Split-Path $cfg -Leaf                      # app.json
Split-Path $cfg -Parent                    # …\config
(Get-Item $cfg).Length                     # size in bytes
```

CODE EXPLAINED

- → `-Filter` — provider-level filter: fastest way to match names
- → `-Depth 1` — limits recursion (implies `-Recurse`)
- → `-PathType Leaf` — `Leaf` = file, `Container` = folder
- → `New-Item -Force` — creates missing parents; no error if the folder exists
- → `Remove-Item -Recurse` — deletes folders and contents. No recycle bin
- → `Join-Path a b c` — multiple child paths need 7+

GET-CHILDITEM OPTIONS

OPTION	EFFECT
`-Recurse / -Depth n`	into subfolders
`-File / -Directory`	only one kind
`-Filter *.log`	fast name match
`-Include / -Exclude`	extra patterns
`-Hidden / -Force`	hidden / everything
`-Name`	names only

PATH CMDLETS

CMDLET	EXAMPLE → RESULT
`Split-Path -Leaf`	a\b.txt → b.txt
`Split-Path -LeafBase`	a\b.txt → b (7+)
`Split-Path -Extension`	→ .txt (7+)
`Resolve-Path`	.\x → full path
`Convert-Path`	PSDrive → real path

NAVIGATION & DRIVES

COMMAND	USE
`Set-Location / cd`	change folder
`Push-Location / Pop-Location`	folder stack
`Get-PSDrive`	list drives
`New-PSDrive -Root`	custom drive

⚠ COMMON MISTAKES

- Using `-Include` without `-Recurse` or a wildcard path. It silently matches nothing; add `-Recurse` or use `-Filter`
- Building paths with `"$dir\$file"`. Breaks on Linux and with trailing slashes; use `Join-Path`
- Running `Remove-Item -Recurse -Force` on a variable that might be empty. Check `Test-Path` and use `-WhatIf` first

✓ TIPS & BEST PRACTICES

- Use `-Filter` before `Where-Object`: it's handled by the file system and is much faster
- Use `-LiteralPath` for names like `report[1].txt`
- Get folder sizes: `(gci $d -Recurse -File | Measure-Object Length -Sum).Sum / 1MB`

QUICK SUMMARY

- `Get-ChildItem -Filter -Recurse -File` — find files
- `Test-Path` / `New-Item -Force` — check and create
- `Copy- / Move- / Rename- / Remove-Item` — manage items
- `Join-Path` / `Split-Path` — portable path handling

Reading & Writing Files

OVERVIEW

`Get-Content` reads a text file as an **array of lines** (or one string with `-Raw`). `Set-Content` writes, `Add-Content` appends, and `Out-File` / `>` write formatted output. PowerShell 7 writes **UTF-8 without BOM** by default; 5.1 uses different defaults per cmdlet, so pass `-Encoding` there.

NOTES

- Lines lose their newline characters when read
- `Get-Content -Raw` keeps the file as one string
- 5.1: `Out-File` / `>` write UTF-16; `Set-Content` writes ANSI
- Use .NET `[IO.File]` methods for very large files

READ, WRITE, SEARCH

```
$lines = Get-Content .\servers.txt          # string[]
$text  = Get-Content .\config.json -Raw     # one string
Get-Content .\app.log -Tail 20              # last 20 lines
Get-Content .\app.log -Tail 0 -Wait         # follow, like tail -f

Set-Content .\out.txt -Value 'line 1', 'line 2'
Add-Content .\out.txt -Value 'line 3'
Get-Process | Out-File .\procs.txt -Width 200

Set-Content .\a.txt 'x' -Encoding utf8BOM  # for old Windows tools

Select-String -Path .\*.log -Pattern 'ERROR' -Context 1, 2
Select-String -Path .\app.log -Pattern 'id=(\d+)' |
    ForEach-Object { $_.Matches[0].Groups[1].Value }
[IO.File]::ReadLines("$PWD\huge.log") | Select-Object -First 3
```

CODE EXPLAINED

- → `Get-Content` — streams lines; each line also has `ReadCount` metadata
- → `-Tail 0 -Wait` — watch a log for new lines; Ctrl+C to stop
- → `Set-Content -Value` — writes strings as-is (no table formatting)
- → `Out-File -Width` — writes what you'd see on screen; widen to avoid cut-offs
- → `Select-String -Context` — lines before/after each match, like grep -B/-A
- → `[IO.File]::ReadLines()` — lazy reader; needs a full path

GET-CONTENT OPTIONS

OPTION	EFFECT
`-Raw`	one string
`-TotalCount n`	first n lines
`-Tail n`	last n lines
`-Wait`	keep reading new lines
`-ReadCount 1000`	batch lines (faster)

WRITING CHOICES

COMMAND	WRITES
`Set-Content`	strings, overwrite
`Add-Content`	strings, append
`Out-File / >`	formatted view
`Out-File -Append / >>`	append view
`Export-Csv / ConvertTo-Json`	data

-ENCODING VALUES

VALUE	NOTES
`utf8NoBOM`	7 default
`utf8BOM`	explicit BOM (7)
`utf8`	BOM in 5.1, no BOM in 7
`unicode`	UTF-16LE
`oem`	console code page

⚠ COMMON MISTAKES

- Treating `Get-Content` output as one string. It's an array of lines; `-match` then filters lines. Use `-Raw` for whole-file regex
- Using `Out-File` for data you'll read back. It writes the formatted, possibly truncated view; use `Set-Content` or `Export-Csv`
- Passing a relative path to `[IO.File]` methods. .NET uses the process directory, not `$PWD`

✓ TIPS & BEST PRACTICES

- Read config files with `Get-Content -Raw | ConvertFrom-Json`
- Use `Get-Content -ReadCount 1000` or `[IO.File]::ReadLines()` for multi-GB logs
- Search a tree: `Get-ChildItem -Recurse -Filter *.cs | Select-String 'TODO'`

QUICK SUMMARY

- `Get-Content` — lines; `-Raw` one string; `-Tail -Wait` follow
- `Set-Content` / `Add-Content` — write / append text
- `Out-File` / `>` — save formatted output
- `Select-String` — grep for PowerShell

JSON, CSV & XML

OVERVIEW

PowerShell converts structured text into objects and back. **JSON** and **CSV** round-trip with paired `ConvertTo-` / `ConvertFrom-` and `Import-` / `Export-` cmdlets; **XML** is cast with `[xml]` and navigated with dots. **CLIXML** saves full PowerShell objects, types included.

NOTES

- `ConvertTo-Json` default `-Depth` is 2; deeper data is flattened
- Every CSV value comes back as a **string**
- 7 drops the CSV `#TYPE` line; 5.1 needs `-NoTypeInformation`
- `-AsHashtable` (6+) gives a hashtable instead of PSCustomObject

ROUND-TRIPPING DATA

```
$cfg = Get-Content .\config.json -Raw | ConvertFrom-Json
$cfg.database.port                          # 5432
$cfg | ConvertTo-Json -Depth 10 | Set-Content .\config.json

@{ name = 'web'; tags = 'a', 'b' } | ConvertTo-Json -Compress
# {"name":"web","tags":["a","b"]}

$users = Import-Csv .\users.csv             # one object per row
$users | Where-Object { [int]$_.Age -gt 30 }
$users | Export-Csv .\out.csv -NoTypeInformation
'a,b', '1,2' | ConvertFrom-Csv              # from strings

[xml]$doc = Get-Content .\web.config -Raw
$doc.configuration.appSettings.add | Where-Object key -eq 'Mode'
Select-Xml -Xml $doc -XPath '//add[@key="Mode"]'

Get-Process | Export-Clixml .\procs.xml  # full objects
```

CODE EXPLAINED

- → `ConvertFrom-Json` — JSON text → objects; use `-Raw` so it gets one string
- → `-Depth 10` — prevents nested objects becoming "System.Collections…" text
- → `-Compress` — single-line JSON
- → `[int]$_.Age` — CSV fields are text; cast before numeric comparisons
- → `[xml]` — parses the text into an XmlDocument you can dot into
- → `Export-Clixml` — keeps types; re-import with `Import-Clixml`

JSON CMDLETS

CMDLET / OPTION	EFFECT
`ConvertTo-Json`	objects → JSON
`-Depth n`	nesting (default 2)
`-Compress`	minified
`ConvertFrom-Json`	JSON → objects
`-AsHashtable`	hashtables (6+)
`Test-Json`	validate (6.1+)

CSV CMDLETS

CMDLET / OPTION	EFFECT
`Import-Csv`	file → objects
`Export-Csv`	objects → file
`-Delimiter ';'`	custom separator
`-Header a,b`	supply column names
`-Append`	add rows

OTHER FORMATS

COMMAND	USE
`Import-PowerShellDataFile`	.psd1 → hashtable
`Export-Clixml`	objects + types
`Import-Clixml`	restore objects
`ConvertTo-Html`	HTML table

⚠ COMMON MISTAKES

- Forgetting `-Depth` on `ConvertTo-Json`. Nested data silently becomes type-name strings (7.1+ warns)
- Comparing CSV numbers as text: `'100' -gt '9'` is False. Cast fields with `[int]` or `[double]`
- Piping `Format-Table` into `Export-Csv`. You export formatting objects; select properties instead

✓ TIPS & BEST PRACTICES

- Use `ConvertTo-Json -Depth 10` as a habit for config files
- Use `Export-Clixml` with `Get-Credential` to store a credential encrypted for the current user (Windows)
- Use `Import-Csv | Group-Object Dept -AsHashTable` to build lookups from spreadsheets

QUICK SUMMARY

- `ConvertFrom-Json` / `ConvertTo-Json -Depth` — JSON round-trip
- `Import-Csv` / `Export-Csv` — CSV rows as objects
- `[xml]` / `Select-Xml` — parse and query XML
- `Export-Clixml` — save full objects

Using Modules & the PowerShell Gallery

OVERVIEW

A **module** is a package of commands. Modules in any folder on `$env:PSModulePath` **autoload** the first time you use one of their commands, so explicit `Import-Module` is rarely needed. Get new modules from the **PowerShell Gallery** with `Install-PSResource` (PSResourceGet, built into 7.4+) or the older `Install-Module`.

NOTES

- Install for yourself with `-Scope CurrentUser`: no admin needed
- `Install-PSResource` defaults to CurrentUser scope
- PSGallery is untrusted by default, so you get a prompt
- 5.1 and 7 have separate user module folders

FINDING, INSTALLING, LOADING

```
Get-Module                                  # loaded in this session
Get-Module -ListAvailable Az*               # installed on disk
$env:PSModulePath -split [IO.Path]::PathSeparator

Find-PSResource -Name ImportExcel           # search the Gallery
Install-PSResource -Name ImportExcel        # CurrentUser by default
Install-PSResource Pester -Version '[5.0,6.0)'   # version range
Update-PSResource ImportExcel
Get-InstalledPSResource
Set-PSResourceRepository -Name PSGallery -Trusted   # no prompt

Install-Module Az -Scope CurrentUser        # PowerShellGet v2 style
Import-Module ActiveDirectory               # explicit import
Import-Module .\MyTools -Force              # reload after edits
Get-Command -Module ImportExcel

#Requires -Modules @{ ModuleName = 'Az.Accounts'; ModuleVersion = '2.0' }
```

CODE EXPLAINED

- → `Get-Module -ListAvailable` — everything installed, loaded or not
- → `PSModulePath` — folders searched for modules, in order
- → `Install-PSResource` — PSResourceGet; faster successor to PowerShellGet
- → `'[5.0,6.0)'` — NuGet version range: ≥ 5.0 and < 6.0
- → `-Force` — reimport so code changes take effect
- → `#Requires -Modules` — fail early if a module or version is missing

PSRESOURCEGET (7.4+)

CMDLET	USE
`Find-PSResource`	search
`Install-PSResource`	install
`Update-PSResource`	upgrade
`Uninstall-PSResource`	remove
`Get-InstalledPSResource`	list installed
`Save-PSResource`	download for offline

POWERSHELLGET V2 (5.1)

CMDLET	USE
`Find-Module`	search
`Install-Module`	install
`Update-Module`	upgrade
`Uninstall-Module`	remove
`Get-InstalledModule`	list installed

WIDELY USED MODULES

MODULE	FOR
`Az`	Azure
`Microsoft.Graph`	Microsoft 365 / Entra
`Pester`	testing
`PSScriptAnalyzer`	linting
`ImportExcel`	Excel without Excel
`dbatools`	SQL Server

⚠ COMMON MISTAKES

- Running `Install-Module` without `-Scope CurrentUser` in 5.1. It needs admin for AllUsers and fails otherwise
- Editing a module and re-running `Import-Module` without `-Force`. The old version stays loaded
- Installing a module in 5.1 and expecting pwsh to see it. The user module folders differ

✓ TIPS & BEST PRACTICES

- Pin versions in scripts with `#Requires -Modules` so upgrades don't break you
- Use `Save-PSResource` to vet a module's code before installing it
- Use `Get-Command -Module X` to see what a new module gives you

QUICK SUMMARY

- `$env:PSModulePath` — where modules autoload from
- `Install-PSResource` / `Install-Module` — get from PSGallery
- `Get-Module -ListAvailable` — what's installed
- `#Requires -Modules` — declare dependencies

Writing Modules

OVERVIEW

A script module is a `.psm1` file in a folder **with the same name**, optionally with a `.psd1` **manifest** describing version, author, dependencies and exports. Keep one function per file in `Public` and `Private` folders and dot-source them from the .psm1. Place the folder on `$env:PSModulePath` and it autoloads.

NOTES

- Folder name = module name = file base name
- Without exports, every function in the .psm1 is public
- List exports explicitly; `*` slows autoloading
- Bump `ModuleVersion` on every release

MODULE LAYOUT & MANIFEST

```
# MyTools\                          ← folder on $env:PSModulePath
#   MyTools.psd1  MyTools.psm1
#   Public\Get-Thing.ps1   Private\Format-Thing.ps1

# --- MyTools.psm1 ---
$public  = @(Get-ChildItem $PSScriptRoot\Public\*.ps1)
$private = @(Get-ChildItem $PSScriptRoot\Private\*.ps1)
foreach ($file in $public + $private) { . $file.FullName }
Export-ModuleMember -Function $public.BaseName

# --- create the manifest ---
$m = @{
    Path = '.\MyTools\MyTools.psd1'; RootModule = 'MyTools.psm1'
    ModuleVersion = '1.0.0'; Author = 'Bob'
    FunctionsToExport = 'Get-Thing'; PowerShellVersion = '7.2'
}
New-ModuleManifest @m
Test-ModuleManifest .\MyTools\MyTools.psd1
Publish-PSResource -Path .\MyTools -ApiKey $env:PSGALLERY_KEY
```

CODE EXPLAINED

- → `folder layout` — Public = exported, Private = helpers
- → `. $file.FullName` — dot-source each function into module scope
- → `$public.BaseName` — file names double as function names
- → `RootModule` — the .psm1 the manifest loads
- → `FunctionsToExport` — what users see; also drives autoload
- → `Test-ModuleManifest` — validates the .psd1 before publishing

KEY MANIFEST FIELDS

FIELD	PURPOSE
`RootModule`	main .psm1
`ModuleVersion`	semver
`GUID`	unique ID (auto)
`FunctionsToExport`	public functions
`RequiredModules`	dependencies
`PowerShellVersion`	minimum version

CONTROLLING EXPORTS

MECHANISM	EFFECT
`Export-ModuleMember`	in .psm1
`FunctionsToExport`	in .psd1 (wins)
`AliasesToExport`	aliases
`VariablesToExport`	variables
`CmdletsToExport`	binary cmdlets

INSTALL LOCATIONS

SCOPE	FOLDER (WINDOWS)
`CurrentUser 7`	Documents\PowerShell\Modules
`CurrentUser 5.1`	Documents\WindowsPowerShell\Modules
`AllUsers 7`	Program Files\PowerShell\Modules
`Linux/macOS`	~/.local/share/powershell/Modules

⚠ COMMON MISTAKES

- Folder and .psm1 names don't match. `Import-Module` by name then can't find the module
- Leaving `FunctionsToExport = '*'`. Works, but autoload must load the module to find commands
- Exporting in the .psm1 but not the .psd1. The manifest's list wins, so the function stays hidden

✓ TIPS & BEST PRACTICES

- Use `Import-Module .\MyTools -Force -Verbose` while developing to see what loads
- Keep tests next to code: `Tests\Get-Thing.Tests.ps1`
- Store the Gallery API key in a secret vault, never in the script

QUICK SUMMARY

- `.psm1` in a same-named folder — a module
- `.psd1` via `New-ModuleManifest` — metadata + exports
- Public/Private + dot-sourcing — tidy layout
- `Publish-PSResource` — release to a gallery

Regular Expressions

OVERVIEW

PowerShell uses the .NET regex engine. `-match` tests a string and fills `$Matches` with the **first** match; `[regex]::Matches()` or `Select-String -AllMatches` find **all** matches. Regex operators are case-insensitive by default; use the `c` versions (`-cmatch`, `-creplace`) when case matters.

NOTES

- Always single-quote patterns so `$` isn't expanded
- `-match` on an array filters; it doesn't set `$Matches`
- Named groups: `(?<year>\d{4})` → `$Matches.year`
- Escape user text with `[regex]::Escape()`

MATCHING, CAPTURING, REPLACING

```
$log = '2026-10-06 14:05 ERROR [db] Timeout after 30s'

if ($log -match '^(?<date>\S+) (?<time>\S+) (?<level>\w+)') {
    $Matches.level                      # ERROR
    $Matches.date                       # 2026-10-06
}
$log -replace '(\d+)s$', '$1 seconds'    # …after 30 seconds
$log -replace '(?<y>\d{4})-(?<m>\d\d)', '${m}/${y}'   # 10/2026-06 …

[regex]::Matches('a1 b22 c333', '\d+').Value   # 1 22 333
Select-String -Path *.log -Pattern 'user=(\w+)' -AllMatches |
    ForEach-Object { $_.Matches.Groups[1].Value }

switch -Regex ($line) {
    '^WARN'          { $warn++ }
    '^ERROR (\d+)'   { "code $($Matches[1])" }
}
'a-b_c d' -split '[-_ ]'                # a b c d
```

CODE EXPLAINED

- → `(?<level>\w+)` — named capture group
- → `$Matches.level` — read a named group from the last scalar match
- → `'$1 seconds'` — numbered group in the replacement (single quotes!)
- → `'${m}/${y}'` — named groups in the replacement
- → `[regex]::Matches()` — every match; `.Value`, `.Groups`, `.Index`
- → `-AllMatches` — Select-String finds more than one match per line

PATTERN SYNTAX

TOKEN	MATCHES
`\d \w \s`	digit, word char, space
`\D \W \S`	the opposites
`.`	any char except newline
`^ $`	start / end
`\b`	word boundary
`[a-z] [^0-9]`	set / negated set

QUANTIFIERS & GROUPS

TOKEN	MEANING
`* + ?`	0+, 1+, 0 or 1
`{3} {2,5}`	exactly / range
`*? +?`	lazy (shortest)
`(…)`	capture group
`(?:…)`	group, no capture
`a|b`	either

REGEX TOOLS

TOOL	RETURNS
`-match`	bool + $Matches
`-replace`	new string
`-split`	array of pieces
`Select-String`	MatchInfo objects
`[regex]::Matches()`	all matches
`(?i) / (?m)`	inline options

⚠ COMMON MISTAKES

- Double-quoting a pattern or replacement with `$1`. PowerShell expands it first; use single quotes
- Expecting `-match` to return all matches. It stops at the first; use `[regex]::Matches()`
- Using a dot for a literal period. `.` matches anything; write `\.`

✓ TIPS & BEST PRACTICES

- Build complex patterns with `(?x)` and comments on separate lines in a here-string
- Test patterns quickly: `'sample' -match 'pattern'; $Matches`
- Use `Select-String -Pattern … -Context 2` to see surrounding lines

QUICK SUMMARY

- `-match` — test + `$Matches` (first match)
- `-replace` — regex substitution with `$1` / `${name}`
- `[regex]::Matches()` — all matches
- `Select-String` / `switch -Regex` — files and branching

Dates & Times

OVERVIEW

`Get-Date` returns a `[datetime]` object with properties (`Year`, `DayOfWeek`) and methods (`AddDays()`). Subtracting two dates gives a `[timespan]`. Use `-Format` for text output, and `ParseExact()` when reading dates in a known format so the result doesn't depend on the machine's culture.

NOTES

- `-Format` returns a **string**, not a date
- Format codes are case-sensitive: `MM` month, `mm` minute
- Date math: `$end - $start` → TimeSpan
- Store and compare in UTC; display in local time

WORKING WITH DATES

```
Get-Date                                # Tuesday, October 6, 2026 9:41 AM
Get-Date -Format 'yyyy-MM-dd HH:mm'     # 2026-10-06 09:41
Get-Date -Format FileDateTime           # 20261006T0941075123
(Get-Date).AddDays(-30).Date            # midnight 30 days ago
(Get-Date).DayOfWeek                    # Tuesday
[datetime]::DaysInMonth(2026, 2)        # 28

$d = [datetime]::ParseExact('06/10/2026', 'dd/MM/yyyy', $null)
$d.Month                                # 10

$span = New-TimeSpan -Start '2026-01-01' -End (Get-Date)
$span.Days                              # 278
((Get-Date) - $file.LastWriteTime).TotalHours

(Measure-Command { Get-ChildItem C:\ -Recurse }).TotalSeconds
Start-Sleep -Milliseconds 500
(Get-Date).ToUniversalTime().ToString('o')   # ISO 8601 UTC
```

CODE EXPLAINED

- → `-Format` — custom .NET format string or a named format
- → `.Date` — same day at 00:00
- → `ParseExact(text, fmt, $null)` — `$null` = current culture; the format fixes the field order
- → `New-TimeSpan` — difference between two dates
- → `.TotalHours` — whole span as a decimal; `.Hours` is just the hour part
- → `Measure-Command` — times a script block; output is suppressed

FORMAT CODES

CODE	EXAMPLE
`yyyy / yy`	2026 / 26
`MM / MMM / MMMM`	10 / Oct / October
`dd / ddd`	06 / Tue
`HH / hh tt`	14 / 02 PM
`mm / ss / fff`	minute / sec / ms
`zzz`	+02:00 offset

USEFUL MEMBERS

MEMBER	RETURNS
`.AddDays(n)`	new date (also Hours, Months…)
`.Date`	midnight
`.DayOfYear`	1–366
`[datetime]::Today`	today 00:00
`[datetime]::UtcNow`	now in UTC
`.ToLocalTime()`	UTC → local

TIMESPAN

EXPRESSION	VALUE
`New-TimeSpan -Hours 2`	02:00:00
`[timespan]'1:30:00'`	1h 30m
`$ts.TotalMinutes`	90
`(Get-Date) + $ts`	date later
`[timespan]::FromDays(7)`	7.00:00:00

⚠ COMMON MISTAKES

- Using `mm` for month. `mm` is minutes; months are `MM`
- Comparing `Get-Date -Format` output as dates. It's text; compare `[datetime]` objects instead
- Parsing `'06/10/2026'` with `[datetime]`. Casts assume US month/day order; use `ParseExact`

✓ TIPS & BEST PRACTICES

- Use `Get-Date -Format FileDate` or `'yyyyMMdd'` in file names; they sort correctly
- Use `.TotalSeconds` / `.TotalDays` for spans, not `.Seconds` / `.Days`
- Use `Get-Date -AsUTC` (7.1+) for timestamps in logs

QUICK SUMMARY

- `Get-Date -Format 'yyyy-MM-dd'` — text output
- `.AddDays()` / `.Date` — date arithmetic
- `New-TimeSpan` / `$a - $b` — durations
- `Measure-Command` / `Start-Sleep` — timing

Jobs & Parallelism

OVERVIEW

PowerShell can run work in the background or in parallel. `Start-Job` runs code in a separate process (heavy, but isolated); `Start-ThreadJob` uses a thread in the same process (fast). `ForEach-Object -Parallel` (7.0+) is the easiest way to process a list concurrently. None of them see your variables unless you pass them with `$using:` or `-ArgumentList`.

NOTES

- Default `-ThrottleLimit` for `-Parallel` is 5
- Job output is serialized: objects become property bags
- `-Parallel` blocks run in separate runspaces
- Parallel is only faster for slow I/O or CPU-heavy work

BACKGROUND AND PARALLEL WORK

```
$job = Start-Job { param($p) Get-ChildItem $p -Recurse } -Arg C:\Logs
Get-Job                                  # State: Running / Completed
Wait-Job $job | Out-Null
$files = Receive-Job $job                # get output (once)
Remove-Job $job

$tj = Start-ThreadJob { Invoke-RestMethod https://api.github.com }
Receive-Job $tj -Wait -AutoRemoveJob

$servers = 'web01', 'web02', 'db01', 'db02'
$timeout = 2
$servers | ForEach-Object -Parallel {
    [PSCustomObject]@{ Server = $_
        Up = Test-Connection $_ -Count 1 -Quiet -TimeoutSeconds $using:timeout
    }
} -ThrottleLimit 10

$bg = Get-Process &                      # background operator (7+)
```

CODE EXPLAINED

- → `-ArgumentList` — values passed to the block's `param()`
- → `Receive-Job` — returns output once; `-Keep` to read again
- → `-Wait -AutoRemoveJob` — wait, collect and clean up in one call
- → `$using:timeout` — copies a caller variable into each parallel runspace
- → `-ThrottleLimit 10` — maximum blocks running at once
- → `&` — end a pipeline with `&` to run it as a job

WHICH TO USE?

TOOL	RUNS IN	STARTUP
`Start-Job`	process	slow
`Start-ThreadJob`	thread	fast
`-Parallel`	runspace	fast
`Invoke-Command`	remote	varies

JOB CMDLETS

CMDLET	USE
`Get-Job`	list jobs + state
`Wait-Job`	block until done
`Receive-Job`	collect output
`Stop-Job`	cancel
`Remove-Job`	delete job object
`-Timeout`	Wait-Job limit

PASSING DATA IN

METHOD	WHERE
`$using:var`	-Parallel, remoting, jobs
`-ArgumentList`	Start-Job / ThreadJob
`param($a)`	receives -ArgumentList
`$_`	-Parallel current item

⚠ COMMON MISTAKES

- Referencing an outer variable in a job without `$using:`. It's `$null` inside the job's runspace
- Modifying a shared collection from `-Parallel` blocks. Not thread-safe; output objects and collect them instead
- Forgetting `Receive-Job` or `Remove-Job`. Output sits in memory and finished jobs pile up

✓ TIPS & BEST PRACTICES

- Prefer `Start-ThreadJob` over `Start-Job` unless you need process isolation
- Tune `-ThrottleLimit` to the bottleneck: higher for network waits, ≈ cores for CPU
- Use `-AsJob` on `ForEach-Object -Parallel` to keep the console free

QUICK SUMMARY

- `Start-Job` / `Receive-Job` — background process
- `Start-ThreadJob` — lightweight background thread
- `ForEach-Object -Parallel` — concurrent pipeline (7+)
- `$using:` — pass outer variables in

Web Requests & REST APIs

OVERVIEW

`Invoke-RestMethod` calls an API and **parses the response** (JSON or XML) into objects, so you can use properties right away. `Invoke-WebRequest` returns the raw response: status code, headers, content, links. Use it for downloads, scraping, or when you need the status and headers.

NOTES

- JSON responses become `PSCustomObject`s automatically
- Send JSON with `ConvertTo-Json` + `-ContentType 'application/json'`
- `-Authentication Bearer -Token` needs a SecureString (7+)
- 5.1 may need TLS 1.2 enabled and `-UseBasicParsing`

CALLING APIS

```
$repo = Invoke-RestMethod https://api.github.com/repos/PowerShell/PowerShell
$repo.stargazers_count                    # star count (a number)

$body = @{ title = 'Bug'; labels = @('ps') } | ConvertTo-Json
$params = @{
    Uri     = 'https://api.example.com/issues'; Method = 'Post'
    Body    = $body; ContentType = 'application/json'
    Headers = @{ Authorization = "Bearer $env:API_TOKEN" }
}
$issue = Invoke-RestMethod @params

$r = Invoke-WebRequest https://example.com
$r.StatusCode                             # 200
Invoke-WebRequest $url -OutFile .\setup.zip   # download

Invoke-RestMethod $api -FollowRelLink    # all pages
try { Invoke-RestMethod $u } catch { $_.Exception.Response.StatusCode }
```

CODE EXPLAINED

- → `Invoke-RestMethod` — GET by default; result is already parsed
- → `ConvertTo-Json` — build the request body from a hashtable
- → `@params` — splatting keeps long API calls readable
- → `-OutFile` — stream the response to a file
- → `-FollowRelLink` — follows `Link: rel=next` paging (7+)

COMMON PARAMETERS

PARAMETER	USE
`-Method`	Get Post Put Patch Delete
`-Body`	string, hashtable, or bytes
`-ContentType`	type of -Body
`-Headers`	extra headers
`-Form`	multipart upload (7+)

POWERSHELL 7 EXTRAS

PARAMETER	EFFECT
`-SkipHttpErrorCheck`	no throw on 4xx/5xx
`-StatusCodeVariable`	save status
`-MaximumRetryCount`	retry failures
`-SkipCertificateCheck`	ignore bad TLS (test only)
`-Authentication`	Basic / Bearer / OAuth

INVOKE-WEBREQUEST RESULT

PROPERTY	HOLDS
`.StatusCode`	200, 404…
`.Content`	body text
`.Headers`	response headers
`.Links`	anchor tags

⚠ COMMON MISTAKES

- Passing a hashtable as `-Body` to a JSON API. It's sent as form data; convert with `ConvertTo-Json` first
- Forgetting `-Depth` on nested request bodies. Inner objects become type-name strings
- Hard-coding tokens in scripts. Read them from `$env:` or SecretManagement

✓ TIPS & BEST PRACTICES

- Use `-SkipHttpErrorCheck -StatusCodeVariable sc` to handle 404s without try/catch
- Inspect a parsed response with `$r | ConvertTo-Json -Depth 5`
- In 5.1, enable TLS 1.2 via `[Net.ServicePointManager]` if HTTPS calls fail

QUICK SUMMARY

- `Invoke-RestMethod` — call an API, get objects
- `Invoke-WebRequest` — raw response, status, downloads
- `-Body (… | ConvertTo-Json)` — send JSON
- `-Headers @{ Authorization = … }` — authenticate

Debugging & Testing

OVERVIEW

Catch bugs at three levels. `Set-StrictMode` turns silent mistakes (undefined variables, missing properties) into errors. The **debugger** (VS Code F5, or breakpoints in the console) lets you step through code. **PSScriptAnalyzer** lints for bad practices, and **Pester** runs automated tests.

NOTES

- Install: `Install-PSResource Pester, PSScriptAnalyzer`
- Windows ships Pester 3.x; current syntax is Pester 5
- Test files are named `*.Tests.ps1`
- In VS Code: F9 breakpoint, F5 run, F10/F11 step

STRICT MODE, BREAKPOINTS, PESTER

```
Set-StrictMode -Version Latest          # undefined vars → error
Set-PSBreakpoint -Script .\deploy.ps1 -Line 42
Set-PSBreakpoint -Variable total -Mode Write   # break when it changes
Invoke-ScriptAnalyzer -Path . -Recurse -Severity Warning, Error

# Get-Greeting.Tests.ps1  (Pester 5)
BeforeAll { . $PSScriptRoot/Get-Greeting.ps1 }
Describe 'Get-Greeting' {
    It 'greets by name' { Get-Greeting -Name Ada | Should -Be 'Hello, Ada!' }
    It 'rejects empty names' { { Get-Greeting -Name '' } | Should -Throw }
    It 'logs the call' {
        Mock Write-Log {}
        Get-Greeting -Name Bob
        Should -Invoke Write-Log -Times 1 -Exactly
    }
}
Invoke-Pester -Output Detailed
```

CODE EXPLAINED

- → `-Version Latest` — strictest checks; scoped to the script
- → `-Mode Write` — breaks whenever the variable is assigned
- → `BeforeAll` — loads the code under test before the tests run
- → `Should -Be` — assertion; failure message shows expected vs actual
- → `{ … } | Should -Throw` — wrap code in a script block to test errors
- → `Mock / Should -Invoke` — replace a command and verify it was called

DEBUGGER COMMANDS

KEY	ACTION
`s / F11`	step into
`v / F10`	step over
`o`	step out
`c / F5`	continue
`k`	call stack
`q`	quit debugging

SHOULD OPERATORS

OPERATOR	PASSES WHEN
`-Be / -BeExactly`	equal / case-sensitive
`-BeNullOrEmpty`	$null or ''
`-Contain`	collection has item
`-Match`	regex matches
`-HaveCount n`	n items
`-Not -Be`	negate any operator

PESTER BLOCKS

BLOCK	PURPOSE
`Describe`	group of tests
`Context`	sub-group / scenario
`It`	one test
`BeforeAll / AfterAll`	setup / teardown
`BeforeEach`	runs before each It
`TestDrive:`	temp folder per test

⚠ COMMON MISTAKES

- Putting code outside `BeforeAll` / `It` in Pester 5. It runs during discovery, not during the test run
- Leaving `Set-StrictMode` off and then chasing a typo like `$Totl`. Strict mode would flag it immediately
- Testing output text instead of objects. Assert on properties so formatting changes don't break tests

✓ TIPS & BEST PRACTICES

- Add `Invoke-ScriptAnalyzer` and `Invoke-Pester -CI` to your build pipeline
- Use `Wait-Debugger` in code that runs in a job or remote session to attach there
- Use `$PSDebugContext` or `Set-PSDebug -Trace 1` for quick line tracing

QUICK SUMMARY

- `Set-StrictMode -Version Latest` — catch silent mistakes
- `Set-PSBreakpoint` / VS Code F9 — pause and inspect
- `Invoke-ScriptAnalyzer` — lint scripts
- `Describe / It / Should / Mock` — Pester tests

Processes, Services & Native Commands

OVERVIEW

Manage running programs with the *-Process cmdlets and Windows services with the *-Service cmdlets. You can also run any **native executable** directly; its text output becomes strings in the pipeline, and its exit code lands in $LASTEXITCODE. PowerShell 7.3+ passes arguments to native programs much more predictably than 5.1.

NOTES

- Service cmdlets are Windows-only
- Native exe output is text: one string per line
- Native exes don't throw on failure; check `$LASTEXITCODE`
- `-Verb RunAs` starts a program elevated (UAC)

PROCESSES, SERVICES, EXECUTABLES

```
Get-Process | Sort-Object WS -Descending | Select-Object -First 5 Name, WS
Get-Process notepad -ErrorAction SilentlyContinue | Stop-Process
Stop-Process -Name chrome -Force
Start-Process notepad .\todo.txt
Start-Process msiexec -ArgumentList '/i app.msi /qn' -Wait -NoNewWindow
Start-Process pwsh -Verb RunAs            # elevated window

Get-Service | Where-Object Status -eq Running
Get-Service -Name Spooler | Restart-Service -Force
Set-Service -Name wuauserv -StartupType Disabled

$out = ipconfig /all                      # string[] of lines
git log --oneline -5
if ($LASTEXITCODE -ne 0) { throw "git failed: $LASTEXITCODE" }
& 'C:\Program Files\7-Zip\7z.exe' a backup.zip .\data
icacls.exe --% C:\Data /grant "Users":(R)   # stop-parsing
```

CODE EXPLAINED

- → `Sort-Object WS` — WS = working set (memory) in bytes
- → `Start-Process -Wait` — waits for exit; `-PassThru` returns the process
- → `-NoNewWindow` — run in the current console
- → `Restart-Service -Force` — also restarts dependent services
- → `$LASTEXITCODE` — exit code of the last native program
- → `--%` — pass the rest of the line to the exe exactly as typed

PROCESS CMDLETS

CMDLET	USE
Get-Process	list / find
Stop-Process	kill (-Force)
Start-Process	launch
Wait-Process	wait for exit
Debug-Process	attach debugger
$PID	this session

SERVICE CMDLETS

CMDLET	USE
Get-Service	list / status
Start- / Stop-Service	run / halt
Restart-Service	stop + start
Set-Service	startup type
New-Service	register exe
Remove-Service	delete (7+)

NATIVE COMMAND TIPS

NEED	DO
Exit code	$LASTEXITCODE
Path with spaces	& 'C:\x y\a.exe'
Capture stderr	exe 2>&1
Throw on failure (7.4+)	$PSNativeCommandUse…
Exact args	--% or Start-Process

⚠ COMMON MISTAKES

- Assuming a failed exe stops your script. It doesn't; check `$LASTEXITCODE` after each important call
- Parsing `tasklist` or `sc query` text. `Get-Process` and `Get-Service` return objects with real properties
- Using `Start-Process` to capture output. It doesn't return stdout; call the exe directly or use `-RedirectStandardOutput`

✓ TIPS & BEST PRACTICES

- Use `Get-CimInstance Win32_Service` to see service paths and accounts
- Use `Get-Process -IncludeUserName` (admin) to see who owns a process
- In 7.4+, set `$PSNativeCommandUseErrorActionPreference` to `$true` so failing exes throw under `Stop`

QUICK SUMMARY

- `Get- / Stop- / Start-Process` — manage programs
- `Get- / Restart- / Set-Service` — manage Windows services
- `exe args` + `$LASTEXITCODE` — native commands
- `--%` — stop-parsing for tricky arguments

Remoting & Credentials

OVERVIEW

PowerShell remoting runs commands on other machines and returns **deserialized objects**. Use `Enter-PSSession` for an interactive shell, `Invoke-Command` to run a script block on one or many computers in parallel, and `New-PSSession` for a reusable connection. Windows uses WinRM by default; PowerShell 7 also supports **SSH** on every OS.

NOTES

- Enable WinRM on a target: `Enable-PSRemoting` (admin)
- Remote output objects have properties but no methods
- Use `$using:var` to send local values to the remote side
- `Get-Credential` returns a PSCredential (SecureString password)

REMOTE COMMANDS

```
Enter-PSSession -ComputerName srv01      # [srv01]: PS> …

Invoke-Command -ComputerName srv01, srv02 { Get-Service Spooler }

$days = 7
Invoke-Command srv01 { Get-ChildItem C:\Logs |
    Where-Object LastWriteTime -lt (Get-Date).AddDays(-$using:days) }

$cred = Get-Credential CONTOSO\admin
$s = New-PSSession -ComputerName srv01 -Credential $cred
Invoke-Command -Session $s { $env:COMPUTERNAME }
Copy-Item .\app.zip -Destination C:\Deploy -ToSession $s

Enter-PSSession -HostName ubuntu01 -UserName bob    # SSH (7+)
$pw = Get-Secret -Name ApiKey -AsPlainText          # SecretManagement
```

CODE EXPLAINED

- → `Enter-PSSession` — interactive; the prompt shows the remote computer
- → `-ComputerName a, b` — runs on all targets in parallel (fan-out)
- → `$using:days` — local variable value used remotely
- → `-Credential $cred` — connect as another account
- → `-ToSession` — copy files over an existing session
- → `-HostName` — SSH transport instead of WinRM

REMOTING CMDLETS

CMDLET	USE
`Enter-PSSession`	interactive remote shell
`Invoke-Command`	run block, get output
`New-PSSession`	persistent connection
`Get-PSSession`	list sessions
`Test-WSMan`	is WinRM reachable?

CREDENTIALS & SECRETS

COMMAND	USE
`Get-Credential`	prompt for user/password
`Export-Clixml`	encrypted file (Windows)
`Get-Secret / Set-Secret`	read / store in a vault
`Register-SecretVault`	add a vault

WINRM VS SSH

	WINRM / SSH
`Platforms`	Windows / all
`Parameter`	-ComputerName / -HostName
`Port`	5985-5986 / 22
`Auth`	Kerberos, NTLM / keys, password

⚠ COMMON MISTAKES

- Using a local variable in a remote block without `$using:`. It's `$null` on the remote machine
- Calling methods on returned objects. They're deserialized copies; run the method inside the script block
- Storing passwords in plain text. Use SecretManagement, or `Export-Clixml` for per-user encrypted files

✓ TIPS & BEST PRACTICES

- Use `Invoke-Command -ComputerName (Get-Content servers.txt)` for fleet-wide checks
- Reuse a `New-PSSession` for several commands; connecting is the slow part
- Install the SecretManagement and SecretStore modules for a local vault

QUICK SUMMARY

- `Enter-PSSession` — interactive remote shell
- `Invoke-Command -ComputerName` — run anywhere, in parallel
- `New-PSSession` / `-Session` — reuse a connection
- `Get-Credential` / `Get-Secret` — handle credentials safely

Windows Management

OVERVIEW

On Windows, PowerShell reaches every part of the OS. The **registry** is a drive you navigate with the same `*-Item` cmdlets as files. **CIM** (WMI) exposes hardware and OS details. You can also query event logs, schedule tasks, and manage local accounts. Most of this sheet is **Windows-only**.

NOTES

- Registry values are *properties* of keys: use `*-ItemProperty`
- Use `Get-CimInstance`, not the old `Get-WmiObject` (gone in 7)
- Many commands need an elevated (admin) session
- `Get-WinEvent -FilterHashtable` is far faster than `Where-Object`

REGISTRY, CIM, EVENTS, TASKS

```
Get-ChildItem HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall
Get-ItemProperty HKCU:\Environment                    # all values of a key
Get-ItemPropertyValue HKCU:\Environment -Name TEMP
Set-ItemProperty HKCU:\Software\MyApp -Name Mode -Value 'prod'

Get-CimInstance Win32_OperatingSystem |
    Select-Object Caption, Version, LastBootUpTime
Get-CimInstance Win32_LogicalDisk -Filter "DriveType=3"

Get-WinEvent -FilterHashtable @{
    LogName = 'System'; Level = 2; StartTime = (Get-Date).AddDays(-1)
} -MaxEvents 20                                       # errors, last 24h

$action  = New-ScheduledTaskAction -Execute pwsh -Argument '-File C:\bk.ps1'
$trigger = New-ScheduledTaskTrigger -Daily -At 2am
Register-ScheduledTask -TaskName Backup -Action $action -Trigger $trigger
Get-LocalUser | Where-Object Enabled
```

CODE EXPLAINED

- `HKLM:\…\Uninstall` — subkeys list installed programs
- `Get-ItemPropertyValue` — a single value, without PS* noise
- `Win32_OperatingSystem` — CIM class: OS name, version, boot time
- `Level = 2` — 1 critical, 2 error, 3 warning, 4 info
- `New-ScheduledTaskAction` — what to run; trigger says when
- `Get-LocalUser` — local accounts (Microsoft.PowerShell.LocalAccounts)

REGISTRY CMDLETS

CMDLET	ACTS ON
`Get-ChildItem HKLM:\x`	subkeys
`New-Item`	create key
`Get-ItemProperty`	read values
`Set-ItemProperty`	write value
`New-ItemProperty -Type`	typed value
`Remove-ItemProperty`	delete value

HANDY CIM CLASSES

CLASS	INFO
`Win32_OperatingSystem`	OS, uptime, memory
`Win32_ComputerSystem`	model, domain
`Win32_LogicalDisk`	drives, free space
`Win32_Service`	services + paths
`Win32_BIOS`	serial number

MORE ADMIN COMMANDS

COMMAND	USE
`Get-HotFix`	installed updates
`Get-NetIPAddress`	IP config
`Test-NetConnection`	ping / port check
`Get-Acl / Set-Acl`	permissions
`Get-Counter`	perf counters

⚠ COMMON MISTAKES

- Using `Get-WmiObject`. It's removed in PowerShell 7; `Get-CimInstance` is the replacement
- Querying `Win32_Product` to list software. It's slow and can trigger MSI repairs; read the Uninstall registry keys
- Filtering events with `Get-WinEvent | Where-Object`. It reads the entire log first; use `-FilterHashtable`

✓ TIPS & BEST PRACTICES

- Run `Get-CimInstance -ClassName Win32_* -List` to discover CIM classes
- Use `Test-NetConnection host -Port 443` instead of telnet for port checks
- Back up a registry key before editing: `reg export HKCU\Software\MyApp app.reg`

QUICK SUMMARY

- `HKLM:` / `HKCU:` + `*-ItemProperty` — registry
- `Get-CimInstance Win32_*` — system info
- `Get-WinEvent -FilterHashtable` — event logs
- `Register-ScheduledTask` — automation on a schedule

Sheet 41

Quick Reference — Language

discovery · variables & automatic vars · strings · operators · control flow · arrays & hashtables · functions & params · errors

DISCOVERY

COMMAND	DOES	#
`Get-Command *item*`	find commands	02
`Get-Help Get-Item -Examples`	examples	02
`Get-Member (gm)`	properties & methods	02
`Get-Alias ls`	what an alias runs	02
`$PSVersionTable`	version info	01
`Verb-Noun -Param value`	cmdlet shape	02

VARIABLES

SYNTAX	DOES	#
`$x = 5`	assign	06
`[int]$n = "42"`	typed / cast	08
`$env:PATH`	environment variable	06
`$script:x / $global:x`	scope modifiers	06
`$_ / $PSItem`	current pipeline object	07
`$? / $LASTEXITCODE`	last success / exit code	07

STRINGS

SYNTAX	DOES	#
`"Hi $name"`	expands variables	09
`'literal $x'`	no expansion	09
`"$($u.Name)"`	subexpression	09
`@" … "@`	here-string	09
`"{0:N2}" -f 3.14159`	format operator	09
`` `n `t ``	escapes (backtick)	09

COMPARISON

OPERATOR	DOES	#
`-eq -ne`	equal / not equal	12
`-gt -ge -lt -le`	greater / less	12
`-like "a*"`	wildcard match	12
`-match "^\d+$"`	regex, fills `$Matches`	12
`-contains / -in`	collection membership	12
`-ceq / -cmatch`	case-sensitive forms	12

OTHER OPERATORS

OPERATOR	DOES	#
`-and -or -not !`	logical	13
`?? / ??=`	null-coalescing (7+)	13
`?. / ?[]`	null-conditional (7.1+)	13
`a ? b : c`	ternary (7+)	13
`1..10`	range	13
`-replace "a","b"`	regex replace	10

CONTROL FLOW

SYNTAX	DOES	#
`if ($x) { } elseif { }`	branch	14
`switch ($v) { 1 {…} }`	switch	14
`switch -Regex / -Wildcard`	pattern cases	14
`foreach ($i in $list) { }`	loop statement	15
`for ($i=0; $i -lt 3; $i++)`	counter loop	15
`while / do { } until`	condition loops	15

ARRAYS & HASHTABLES

SYNTAX	DOES	#
`@(1, 2, 3)`	array	24
`$a += 4`	copy + append (slow)	24
`[List[int]]::new()`	growable list	24
`@{ Name = "x"; Age = 3 }`	hashtable	25
`[ordered]@{ }`	keeps key order	25
`[pscustomobject]@{ }`	custom object	17

FUNCTIONS

SYNTAX	DOES	#
`function Get-X { param($Name) }`	function	21
`[CmdletBinding()]`	advanced function	22
`[Parameter(Mandatory)]`	required param	22
`[ValidateSet("a","b")]`	allowed values	22
`ValueFromPipeline`	accept piped input	23
`begin { } process { } end { }`	pipeline blocks	23
`SupportsShouldProcess`	`-WhatIf` / `-Confirm`	23

ERRORS

SYNTAX	DOES	#
`try { } catch { } finally { }`	handle	20
`-ErrorAction Stop`	make errors terminating	19
`$ErrorActionPreference`	default behaviour	07
`$_.Exception.Message`	error text in catch	20
`throw "message"`	raise	20
`Write-Error`	non-terminating error	19

⚠ COMMON MISTAKES

- `catch` not firing — the error was non-terminating; add `-ErrorAction Stop`
- `$a += $x` in a big loop — rebuilds the array each time; use a `List`
- Comparing with `=` or `==` instead of `-eq`

✓ TIPS & BEST PRACTICES

- Pipe to `Get-Member` to see what an object really has
- Use `[pscustomobject]` for output, not formatted text
- Add `[CmdletBinding()]` to get `-Verbose`, `-ErrorAction` for free

QUICK SUMMARY

- Verb-Noun cmdlets, objects through the pipeline
- `-eq -like -match -contains` compare
- `@()` arrays, `@{}` hashtables
- `try` / `catch` + `-ErrorAction Stop`

Sheet 42

Quick Reference — Pipeline, Files & System

pipeline cmdlets · formatting · output streams · files & paths · JSON / CSV / XML · modules · jobs · web · processes · remoting

PIPELINE

CMDLET	DOES	#
`Where-Object { $_.CPU -gt 10 }`	filter	16
`ForEach-Object { $_.Name }`	per item	16
`Select-Object Name, Id`	pick properties	16
`Sort-Object CPU -Descending`	sort	16
`Group-Object Status`	group	16
`Measure-Object -Sum`	count / sum	16
`@{N="MB"; E={$_.WS/1MB}}`	calculated property	17

FORMATTING & OUTPUT

CMDLET	DOES	#
`Format-Table -AutoSize`	table view	18
`Format-List *`	all properties	18
`Out-File / >`	write to file	04
`Write-Output`	success stream	04
`Write-Verbose / Write-Warning`	other streams	04
`2>&1 / *>`	redirect streams	04

FILES & PATHS

CMDLET	DOES	#
`Get-ChildItem -Recurse -Filter *.log`	list files	28
`Test-Path $p`	exists?	28
`New-Item -ItemType Directory`	create	28
`Copy-Item / Move-Item`	copy / move	28
`Remove-Item -Recurse -Force`	delete	28
`Join-Path / Split-Path`	build / split paths	28

READ & WRITE

CMDLET	DOES	#
`Get-Content f.txt`	lines	29
`Get-Content -Raw`	whole file	29
`Set-Content / Add-Content`	write / append	29
`ConvertFrom-Json / ConvertTo-Json -Depth 5`	JSON	30
`Import-Csv / Export-Csv -NoTypeInformation`	CSV	30
`[xml](Get-Content f.xml)`	XML	30

MODULES

CMDLET	DOES	#
`Find-Module Name`	search the Gallery	31
`Install-Module -Scope CurrentUser`	install	31
`Import-Module`	load	31
`Export-ModuleMember`	public functions	32
`New-ModuleManifest`	.psd1 file	32

JOBS & WEB

CMDLET	DOES	#
`Start-Job { }`	background job	35
`Receive-Job -Wait`	get results	35
`ForEach-Object -Parallel`	parallel loop (7+)	35
`Invoke-RestMethod $url`	REST → objects	36
`Invoke-WebRequest $url`	raw response	36
`-Method Post -Body ($b | ConvertTo-Json)`	send JSON	36

PROCESSES & SERVICES

CMDLET	DOES	#
`Get-Process / Stop-Process`	processes	38
`Start-Process -Wait`	launch	38
`Get-Service / Restart-Service`	services	38
`& "C:\tool.exe" args`	run native command	38
`Get-CimInstance Win32_OS`	system info (CIM)	40
`Get-EventLog / Get-WinEvent`	event logs	40

REMOTING & TESTING

CMDLET	DOES	#
`Enter-PSSession srv`	interactive remote	39
`Invoke-Command -ComputerName`	run remotely	39
`Get-Credential`	prompt for creds	39
`Set-PSBreakpoint`	debugger	37
`Invoke-Pester`	run Pester tests	37
`Describe / It / Should -Be`	Pester syntax	37

DATES & REGEX

SYNTAX	DOES	#
`Get-Date -Format "yyyy-MM-dd"`	formatted date	34
`(Get-Date).AddDays(-7)`	date math	34
`Select-String -Pattern "err"`	grep files	33
`$Matches[1]`	capture group	33
`[regex]::Matches($s, $p)`	all matches	33

⚠ COMMON MISTAKES

- Format-* in the middle of a pipeline — it outputs formatting objects, not data
- `ConvertTo-Json` without `-Depth` silently truncating nested objects
- `Export-Csv` in Windows PowerShell 5.1 adding a type header (use `-NoTypeInformation`)

✓ TIPS & BEST PRACTICES

- Filter left: use `-Filter` on the source cmdlet before `Where-Object`
- Use `Invoke-RestMethod` — it parses JSON into objects for you
- Install modules with `-Scope CurrentUser` to avoid admin rights

QUICK SUMMARY

- Where / ForEach / Select / Sort — the core pipeline
- `Get-Content` / `Set-Content` for text, `Import-Csv` for data
- `Invoke-RestMethod` for APIs
- `Invoke-Command` for remote work